TABLE OF CONTENT

SL NO:	CONTENT	Page No:
CHAPTER - I	**INTRODUCTION**	1
CHAPTER - II	**SYSTEM ANALYSIS**	9
CHAPTER – III	**REQUIREMENTS SPECIFICATION**	15
CHAPTER - IV	**TECHNOLOGY STACK**	21
CHAPTER - V	**SYSTEM DESIGN**	27
CHAPTER - VI	**DETAILED IMPLEMENTATION**	40
CHAPTER - VII	**PROJECT MODULES**	54
CHAPTER - VIII	**SYSTEM TESTING**	61
CHAPTER - IX	**DEPLOYMENT AND MAINTENANCE**	68
CHAPTER - X	**CONCLUSION & FUTURE SCOPE**	73
	REFERENCES	77
	APPENDIX	78

LIST OF TABLES

Table No.	Title	Page
2.1	Comparative Analysis: Manual vs Proposed System	14
3.1	Hardware Requirements	16
3.2	Software Requirements	16
3.3	Functional Requirements Summary	19
4.1	Technology Comparison: Backend Frameworks	24
4.2	Technology Comparison: Databases	23
5.1	Database Table: admin	26
5.2	Database Table: students	32
5.3	Database Table: hostel_blocks	33
5.4	Database Table: rooms	34
5.5	Database Table: room_applications	34
5.6	Database Table: room_allocations	35
7.1	Module Overview	55
8.1	Student Module Test Cases	63
8.2	Admin Module Test Cases	64
8.3	Security Test Cases	66

LIST OF FIGURES

Figure Number	PARTICULARS	PAGE NUMBERS
5.1	System Architecture Diagram	29
5.2	Data Flow Diagram â€“ Level 0 and Level 1	32
5.3	Use Case Diagram	36
5.4	Entity-Relationship Diagram	37
5.5	Sequence Diagram	38
7.1	Module Interaction Diagram	56
B.1	Home Page Screenshot	90
B.2	Student Login Page Screenshot	91
B.3	Admin Login Page Screenshot	92

CHAPTER 1:

INTRODUCTION

1.1 Project Background

All educational establishments worldwide experience some problems in managing hostel rooms for students. Every year there are thousands of student applicants in search for few available space, but the administrative part of this process has got quite many challenges. They face such tasks like dealing with incoming applications, validating students' qualifications, examining available rooms in different buildings and making an equitable distribution with only manual files and MS Excel spreadsheets. And what can one anticipate from it? It is obvious Long lines in the hostel administration office; lost applications, mixing of rooms and resulting confusion as well as uncertainty among students concerning their place in the dorms.
Our system came into existence after witnessing firsthand how inefficient these operations were at our institution. The hostel office keeps its physical registers and students need to go to the office personally to fill in an application form, ask about their status or encounter any challenges. There exists no digital database in order for the students to see the room availability through their phones and also there is no automatic prevention of any duplicate room allocation. And when the institution saw a fifteen percent increase in enrollment last year, the ongoing outdated manual process started to be overcome by the amount of work.
Hostel Room Allotment System (HRAS) is an online system which is supposed to take over from the manual procedures which lack transparency with a more flexible and effective digital alternative. HRAS gives ease of registration and also a chance for students to apply for accommodations and receive updates on their room allocations using every gadget with access to internet connection. Administrators have got an easy-to-use control panel showing real-time occupancies of rooms; they are able to administer several blocks of rooms, deal with incoming applications and provide required reports from just one window.

1.2Motivation

Several reasons prompted us towards creating the system:
It's operational inconvenience that led us to create the system. At our school the head of the hostel described the annual procedure of assigning students rooms as 'chaotic.' We receive applications in written form, therefore we have to compare these with information about vacant rooms by hand reading through different registers one by one. It becomes very difficult for us to detect errors and corrections cost much of our valuable time.
Students were not satisfied with our system. A random survey involving fifty students residing in the hostel showed that 72% of them went to the hostel administration office more than three times before they could confirm that they had been allocated to rooms. 38% of them stated that they were given rooms that were already fully occupied causing some sort of re-assignment of rooms. Such type of inconveniences damages confidence in the whole administration system of our higher education establishment.
Technology Readiness
Modern web technologies especially lightweight JavaScript frameworks and cloud-friendly database engines have become mature enough that it is possible to create a full-featured management application as part of a student project. Node. js, Express. js, and MySQL offer a stack that is well documented, has high performance and meets the needs of most educational institutions in respect to hostel operations.
Academic learning
This project did more than solve a genuine operational challenge; it allowed us to gain experience in

full-stack web development from requirement analysis up to deployment. It deals with database schema design, development of the RESTful APIs, authentication and authorization, validation of the user input, ensuring the security measures and development of front-end interface that is adaptable to various devices with all the knowledge sets agreed upon within the industry.

1.3 Problem Definition

The main issue can be clearly described like so: there does not exist any kind of digital central system in our establishment whereby it helps monitor and handle room allotment in the hostel starting from student application to room allocation to actual living and recording of facts related to occupation. The lack of a proper system has resulted into other series of complications:
1) Data scattering around. The information about student records, room inventory, and how the rooms were allocated is scattered in many physical registers and manual spreadsheets kept by those in charge. There is high possibility of getting it wrong and wasting so much of one's time while gathering the information.
2) Slow processing speed. In the absence of a digitalized workflow, each application has to be looked into manually, checked against whether there is available room and put down on paper. For instance during peak registration periods this could lead to delay and processing of data may take couple of weeks before completion.
3) Multiple allocations. Manual update of the room register makes real possibility that two students are allocated into the same bed and same room but this would only be known once the two students have really moved in.
4) Lack of clear visibility of allocation to the students. There is no way the student can know the progress of his application for accommodation in the hostel, see what rooms are vacant, and get information regarding allocation without going physically to the hostel administration office.
5) Deficiencies in generation of reports. Making up of reports concerning number of inhabitants, statistics of the hostels or distribution of students to the respective accommodations requires collection of information which is time consuming meaning thereby, reports are prepared occasionally and they are far off date when done.

1. 4 Objectives

The project has an aim of achieving several objectives as stated below:
1. The project is meant to come up with a web application that digitizes how hostel rooms are alloted which should be inclusive of student registration application submission, room allocation and occupation management.
2. The system will have role based access management having different portals for both students and the administration team and each designed to be customised to meet particular roles assigned to every class of users.
3. A well normalized relational database should be constructed in order to express the relationships and concepts that are inherent within the hostel management like those on students, hostel blocks, rooms, applications, allocations, payments, and audit logs.
4. The design of RESTful API is done with strict input validation, error management and appropriate security measures in place such as JWT authentication, password hashing, limit to rate, HTTP security headers.
5. The project should be able to provide live real-time dashboards so that administrators may gain an instant view of important facts such as total number of students, vacant rooms, applications under consideration, and the accommodation percentage of respective hostel blocks.
6. It should enable the students to take care of themselves with an opportunity of registration, making room requests, monitoring on application status, accessing room and flat mates' details and also

managing their profiles– all through the use of web browser.
7. Proactive reporting of some specific issues can be generated such as overview of accommodation by block, spread of students according to course year and gender, and analysis of distribution of types of rooms used.
8. To ensure consistency in the stored information, database constraints, automatically triggered updates of the occupancy counter, and support for execution of transactions over multiple steps are put in place.

1. 5 Scope

The scope of this system is defined and comprises the following main components:
- Management of the whole life cycle of a student: This includes going through the process of enrollment giving out one's academic qualifications and personal details, updation of the profile, password change, and account inactivation among others.
- Maintenance of hostel infrastructure: Creating and managing the whole hostels which are categorized into different blocks and which are associated with certain features such as gender type, wardens, and floors as well as creation and maintenance of the rooms which are identified by their types, capacity, rent, and status among others.
- Processing of applications: Here, students put forward their requests for accommodation stating the preferred location and the kind of house they would like and also indicate the start date they are looking at. Administrators assess and decide whether they accept, decline or put on hold or waiting list of these requests from the students.
- Assignment of rooms: The administrators are supposed to assign students to designated rooms and beds while the system checks if there is any vacancy before assigning them thus preventing double assignment and also automatically update of occupancy counters using the database triggers.
- Follow-up of payment records: Oversee payment of students on rent, security deposit and other related payments due to allocation made and also record the status such as pending, already paid, rejected, or reimbursed.
- Keeping a record of all important activities in the system: All major activities carried out in this system are logged for easy identification including the type of user who carried it out, action performed, tables that were affected, new values, old values, IP address and date stamp are recorded for responsibility purposes and easy diagnosis of the problem.
- To generate of reports: Reports concerning accommodation within hostel, different room types, and also characteristics of students by different courses, years and sex can be produced.

What is out of the scope for this given stage is sending emails, uploading documents like identification proof, developing native mobile applications, integrating payment gateways online and also dealing with many educational institutions. But this is thoroughly discussed in chapter ten regarding future expansion.

1. 6 Applications

The system, though developed for our specific institution, is meant to be flexible enough to accommodate all educational institutions that have in charge of student hostels. A few potential deployment environments consist of:
- The universities and colleges that have accommodations for both undergraduates and postgraduates.
- The boarding schools which have the responsibility of assigning dorms based on class levels and various houses.
- Some training institutions for instance military academies as well as medical residency programs in which they give out shared quarters in connection with their training plans.

- Corporate guest houses where allocation of visitor's rooms in which you follow up with approval from the management is somewhat similar to hostel managing processes too.

The modular structure of the system, where one can configure blocks, rooms, and allocation rules is very modifiable rather than being fixed coding, hence allows for customization to match some diverse institutional setups with minimal programming involved.

CHAPTER 2:

SYSTEM ANALYSIS

2.1 Study of the Existing System

Prior to creating any replacement, it is vital to know just how the ongoing hostel allocation system functions from the very beginning. At our institution, the whole procedure takes place in an almost this way each semester:
Step 1: Annunciation and form circulation
The hostel office prints out some forms in hard copy format and are kept at the administrative table for collection. Notification is spread using various means such as putting up on notice boards (as well as through heads of departments) about the opening of hostels for the future semester for applications. Those who wish to be accommodated must obtain the form in person.
Step 2: Submitting of Forms
Students fill in the form using pen for all data including his/her name, ID no., program, year of study, phone number, permanent residential location, and any individual additional requirements of accommodation based on particular needs (disability, health issues, etc.). The completed form must be submitted back in the hostel office table where a staff member stamps with a receipt number on it.
Step 3: Verification Procedure
In the offices, there is a manual verification process conducted for every single form referring to the total list of enrolled students in our institution. This may involve some level of coordination with either of the examination section, academic office, and so on.
Step 4: Check of Rooms
The warden or a designated officer shall check through the room register- an orderly huge pasteboard volume arranged by block and floor- with a view of establishing the number of rooms having vacant beds. Since the register is updated manually and also sometimes not done instantly, we get the picture of rooms usually at low frequency.
Step 5: Decision on Allotment of Rooms
The warden will have the authority to decide the room allotment, commonly giving priority to far-distance hometowns, senior students, and to all those with certain medical requirements. All these decisions are well registered in the register itself by recording the student's full names and ID numbers adjacent to specific room entries.
Step 6: Informing the Students
The list of allocated students will be displayed in the hostel notice board. The students must check out the board physically or make a phone call to the office to find out if they have got their rooms assigned to them. But there is no proper notification of status provided to the students.
Step 7: Moving In and Updating the Records
Hostel accommodation is given to the students who have been selected for rooms, and then the assigned room keys can be obtained at the office after having settled their securities and one-month rent in advance (through cash or bank draft) whereby they sign an agreement concerning rules for accommodation. All payments received will be recorded in a separate register of payments.
The whole cycle starting from form distribution until the last move-in takes between 3 to 4 weeks in a normal situation of high demand at the beginning of each academic year.

2.2 Weaknesses of the Current System

This manual procedure explained here has a lot of known disadvantages and weaknesses:
1. Data redundancy and inconsistency
The information about students is found in several documents such as application forms, room registries, and payment records with some wardens also maintaining data in excel spreadsheets. When

a student's program of study or phone number change occurs, it doesn't affect all the filed data and thus leading to inconsistencies.

2. Slow processing speed

Every application must undergo verification individually and compared with the available rooms and then recorded on paper one by one. During the admission period, the hostel office receives more than two hundred applications in ten days. The two employees dealing with them must take almost ten minutes per application, thereby consuming their whole day and resulting in multiday delays.

3. Susceptibility to errors

This includes problems like the poor handwriting that may lead to wrong reading of the allocated rooms among other factors that cause discrepancies. In the last academic year of the institution, fourteen cases were experienced where there were more than one student occupying the same bed.

4. Lack of transparency for students

Students do not have any easy means of finding their application status or seeing the status of the room availability. To know what is happening, they have to go to the office personally or they make a call and in many instances, the personnel cannot respond on time. Lack of visibility fosters frustrations as well as constant asking for information thereby increasing additional workload to the office staff.

5. No historical record

Once a particular semester comes to an end and the register is taken to the archive, tracing back the records of a given student's accommodation history needs retrieval of old books from the storehouse. Thus it becomes difficult to come up with reports over extended periods or evaluating demands of hostels across years.

6. Lack of reporting capability

For example making a report on the male or female composition of hostel residents or Block C on the third floor's occupancy level requires that staff look through every page of the register manually. Those reports are rarely prepared and when they are so, they are probably already outdated by the time they are passed to decision makers.

7. Security and accountability concerns

The paper-based records fail to provide inherent audit trail. In case there is any alteration on the room entry whether by error or intentionality, we cannot easily determine who did that change and the moment of intervention. Absence of accountability results in possibilities of inequality and corruption within the accommodation process of the university.

8. Limitations in scalability

As the learning institution expands itself in size and put in place other new halls of residence, such manual system does not accommodate increases amicably. Increase in the number of wings leads to increase in the number of registers necessary, hence more work is done in cross checks and also higher likelihood of mistakes.

2.3 Proposed System

The Hostel Room Allotment System (HRAS) aims at overcoming all the flaws noted before by transforming each stage in the distribution process into digital form. The proposed system is divided into three principal parts:

An online frontend whereby both the learners and administrators can easily use normal web browsers on any gadgets like desktop computer, laptop, tablet, or smartphones. The frontend application runs under the command of HTML5, CSS3, and JavaScript along with the Bootstrap 5 framework for an adjustable layout, which ensures that user interface can change smoothly across different display units. We have three leading portals namely; general public landing page, a student's own end (which includes registration, login, profile view, applying for accommodation and more about you), and administrative side (which is comprised of login part, dashboard section, taking control over students, blocking the room and the room administration, managing applicants, assigning room and generating report).

Node. js/Express. js backend framework that offers RESTful APIs. All actions of the user on the front end such as attempts to log in, applications of rooms, and a request of a report of the room occupancy gets translated to HTTP requests towards the backend API. On the backend side, we manage identification (using JWT tokens), permission control based on roles of users in the system, validation of data entered (by means of express-validator), operational rules (within respective controllers), and interaction with database via the mysql2 module that has implemented pooling. High-end safety is employed in several ways: helmet for HTTP security headers, CORS for controlled cross-origin resource sharing, express-rate-limit for controlling of too many incoming requests per minute to prevent abuse, and bcrypt for password hash in one direction.
There is a well-established MySQL relational database having normalization structure consisting of eight data tables - admin, students, hostel_blocks, rooms, room_applications, room_allocations, payments, and audit_logs- interconnected through links of primary keys. Database schema also contains calculated fields e. g. available_beds depends on capacity and some data taken from occupied_beds), database views for some tough inquiry to be made and there are three triggered functions to automatically adjust the counters for the number of beds allocated or reallocated after an allotment has been done, altered or removed.

2.4 Advantages of the Proposed System

The proposed system has many practical advantages over the manual method:
1) Centralized Data Storage. Every record on the students, room inventory, requests, and assignments and payments are contained within a single MySQL database. So we have just one reliable point, eliminating those differences found in a paper file.
2) Much faster processing time. An administrator can go through a pending application, look out on available space (the system shows real-time availability), and then assign the room in less than two minutes as opposed to ten plus minutes taken up by the manual work. It represents an increase in speed of at least five times.
3) Deletion of double allocation. A system implements a unique constraint on active assignments per student and also checks whether a room is empty before allocating it. When there is no empty room, the system shows an error message and will prevent the allocation attempt. Also, triggers in the database are responsible for correct updation of the count of occupants.
4) Complete transparency for the students. Students can enter into the website anytime from any location, get some related information about the application i. e. whether it is in progress, accepted or declined, placed on waiting list or cancelled, and know very well their designated room data i. e. room number, block and number of the floor level and also their roommate's information, and see what other rooms are available in every section or unit.
5) Complete audit history. Each important operation performed on the system is saved on the audit_logs table including information on who performed what on specific information and at what particular time with both old and new value saved in JSON format. This is very transparent and will help support any kind of dispute analysis that may be necessary in case of such problems arising in future.
6) Reporting on demand. Administrators can print out some information concerning how many people reside in a particular wing, know the division of various type of room and other statistics regarding students categorized on different program level, first/second/final year and gender of students all with ease of button click. The generated report has the latest data from the database, not an outdated version manually put together sometimes ago.
7) Growth in scalability. Even when we plan on increasing our room block or say by adding more than a hundred new room facilities, there will be need of changing nothing on our end in the program development part. It only requires the system administrator to add the new items via the administrative interface. The integrated connection pooling for database access and simple stateless nature of API will ensure the right performance consistency with growing amounts of data.

8) High security level. Passwords are not stored in clear text ever. We use bcrypt hashing with defined salt rounds so that even if the whole database falls into wrong hands, user password remain hidden and safe. The issued JWT tokens will become invalid after certain amount of time and too rapid login attempts are blocked.

2.5 Feasibility Study

2.5.1 Technical Feasibility

Here our main question is whether it's possible to get all the necessary technologies to put up our system. To begin with we will state some technologies we are going to employ, and they are within our reach and knowledge.
A couple of technologies that have been identified include Node. js (which requires node 14 or more advanced, v18 LTS being our case), which is among the commonly used runtime environment for server side JavaScript program which comes with an array of packages at our disposal on the npm store. Then there is Express. js (v4. 18), being the de facto standard for web application programming using Node. js, and also in production it is used by enterprises and even the starting ones. MySQL (v8. 0) is proven database system with many years of usage experience and vast supporting documents, also it supports very smooth interaction with Node. js through mysql2. In addition, we are going to use Bootstrap 5 as our CSS and the plain JavaScript works very well in the latest internet browsers.

2.5.2 Economic Feasibility

All of the listed technology tools have free license of open source type. For carrying out development, one does not require any expensive computer because he can do it on an operating system like Windows, macOS, or Linux with conventional applications that comprise of code editor (Visual Studio Code), terminal, a browser, and also for running MySQL locally XAMPP. As such, the developer who is also the owner of the project can perform his duty without facing any problem due to his skills gained from studies on some topics like JavaScript, Node. js, CSS & HTML, and SQL and so he has a good ground towards completion of the project based on technicality. Thus our project is technically possible. We can go ahead to discuss about the feasibility of this project in economic terms. A thorough examination is carried out concerning both development expenses and future operational benefits or cuts.
The cost of developing our student project is close enough to zero as no payment is supposed to be made for such kind of project to the university. All applied technologies are completely free to use (because there are no licensing fees), the project development was done on my own personal computer, and during development we did not incur any hosting charge since we have our own local server at XAMPP. However if our system was hosted at a cloud provider, such as a small VPS that costs $5 a month, then every year we will pay around $60 – a minimal sum in comparison to the man hours of staff which can be saved via this system. The benefit of reducing man hours of the administrative team is tremendous. Our hostel administration office usually employs some two employees' estimated time of about four weeks for each semester during registration season. These individuals basically dedicate their time to dealing with room assignments paper work. In contrast, with our newly established digital room assignment system we can easily do the same room allocation work within 3 days with just the help of a single individual. Consequently in the entire academic session, we gain approximately 160 working hours in total, and these can be utilized for some other important duties instead of this paper work. On this basis the proposed project appears to be economically viable and financially beneficial.

2.5.3 Operational Feasibility

Operational feasibility assesses whether the system can be realistically adopted by its intended users. To the students the system presents a familiar web interface which includes login form, dashboard, card and table, and drop down menu that do not require any training. In fact the HRAS interface poses no learning curve because students have been using web application in their daily routine which include course registration and online banking etc.
For the administrator the dashboard offers a combined view of important data that does not demand any technical information about the database. All actions such as adding block, creating room, process application can be done via simple web forms containing validation feedback. It will be enough for just one hour training session to make hostel staff perfect with the system.
Infrastructure requirement is very low. The system runs under all current web browsers and there is need for no setup to the client. Server part may be executed in not powerful machine (or even a regular office computer with XAMPP installed). Connection to the internet within university campus is enough; connection via global internet is unnecessary if system is located at institution's intranet.
Due to these factors the project is practically realizable.

2.6 Comparative Analysis

Table 2.1: Comparative Analysis â€“ Manual System vs. Proposed HRAS

Parameter	Manual System	Proposed HRAS
Data Storage	Paper registers, Excel sheets	Centralized MySQL database
Accessibility	Office hours, physical presence	24/7, any device with browser
Processing Speed	~10 minutes per application	~2 minutes per application
Error Rate	High (manual transcription)	Near-zero (validated inputs)
Double Allocation Risk	Significant	Eliminated (DB constraints)
Student Visibility	None (must visit office)	Real-time status tracking
Reporting	Manual, infrequent	On-demand, real-time
Audit Trail	None	Full (audit_logs table)
Scalability	Poor (more blocks = more chaos)	Excellent (config-driven)
Security	Physical lock on registers	JWT, bcrypt, HTTPS, rate limits
Cost to Operate	High (staff time)	Low (automated processes)
Data Backup	Manual photocopies	Automated DB backups

CHAPTER 3:

REQUIREMENTS SPECIFICATION

3.1 Hardware Requirements

The hardware requirements for the Hostel Room Allotment System are intentionally modest. Because the application runs in a web browser on the client side and on a lightweight Node.js server, it does not demand high-end computing resources. The tables below detail the requirements for the server machine (which hosts the backend and database) and the client machines (which access the system through a browser).

Table 3.1: Hardware Requirements â€“ Server

Component	Minimum Requirement	Recommended
Processor	Dual-core CPU, 2.0 GHz	Quad-core CPU, 2.5 GHz+
RAM	4 GB	8 GB
Hard Disk	20 GB free space	50 GB SSD
Network	100 Mbps LAN	1 Gbps LAN
Display	Not required (headless OK)	Optional monitor for setup

Table 3.1b: Hardware Requirements â€“ Client

Component	Minimum Requirement
Processor	Any modern processor (2015+)
RAM	2 GB
Network	Campus Wi-Fi or wired LAN
Display	1024 Ã— 768 resolution
Input	Keyboard and mouse/touchscreen

3.2 Software Requirements

Table 3.2: Software Requirements

Component	Software	Version	Purpose
Operating System	Windows 10/11 or Ubuntu 20.04+	Latest	Server hosting
Runtime Environment	Node.js	14.0+ (18 LTS recommended)	Backend execution
Package Manager	npm	6.0+	Dependency management
Web Server	XAMPP (Apache + MySQL)	8.2+	Local development server
Database Server	MySQL	8.0+	Data storage
Web Browser	Chrome, Firefox, Edge, Safari	Latest	Client access
Code Editor	Visual Studio Code	Latest	Development
API Testing	Postman	Latest	Endpoint testing
Version Control	Git	2.30+	Source code management

3.3 Functional Requirements

Functional requirements define what the system must doâ€”the specific behaviors and operations it must support. These requirements are organized by user role.

3.3.1 Student Functional Requirements

FR-S1: Student Registration
To register, visitors are expected to create student accounts with some information including student ID, student email, passwords, names, and gender and course of study, while also giving a year that you are supposed to be in. It will include more details such as date of birth, phone number, address, department, name of emergency contact person, and the emergency contact number. The system will validate all data inputted in the forms which include for instance student ID that should be 5 to 20 letters or numbers, email has got to follow a given format, password has to have six characters at least comprising one capital letter, one small letter, and a digit. Following through with a successful sign up, it is supposed to return a JWT access token and a refresh token.
FR-S2: Student Login
A registered student should be able to get into the system with his/her student ID and passwords. In this case, the system has to authenticate the given user credentials from our database and ascertain whether the given account is active. In that case, we should update the last login time stamp too and provide JWT tokens on successful login. With respect to failed authentication, it is supposed to provide a generalized error message that should help prevent exposing to the outside world the fact that student ID exists in our database.
FR-S3: Profile Management
Upon successful verification and authentication, the student who is verified will be capable enough to see on his profile all his details and in addition, modify certain variables that can be modified which are first name, last name, phone, address, and the emergency contacts. Nonetheless, our system does not allow the alteration of student ID, gender, course, or emails within the profile update URL.
FR-S4: Password Change
An already authenticated student should have a right to change his/her password whereby s/he is required to give the old password and set a new one. Our system validates the old password before allowing the changing process of the password.
FR-S5: Room Application
On signing in successfully as an authenticated student, s/he can easily fill in his application form seeking accommodation in a dormitory indicating preferences among them the desired block, room type like single, double, triple, quadruple, or other types of rooms that he/she would love to have. S/he will indicate period of stay in terms of number of months as well as list any additional needs that s/he might have. Nevertheless, our system cannot take his application for consideration if s/he had submitted another application which is either pending or approved or s/he occupies an allocated room right now.
FR-S6: Application Tracking
The logged-in student is supposed to have an opportunity of looking at a list of all his applications whereby they are placed in order and s/he can also call up details of every particular application which includes administrative comments as well as certain important information about the process followed so far and even have a chance to cancel a given pending application.
FR-S7: Allocation Viewing
It is expected that after verifying and authenticating himself, a student who is confirmed can easily see his current active room allocation (room number, block, floor, type, available facilities, warden contacts, roommates’ details) and also look at his history in the allocation.
FR-S8: Dashboard. An authenticated student is expected to see on the dashboard information about themselves including their profile, number of applications that are active with their allocation for now and a payment summary.
FR-S9: Room Browsing. An authenticated student should see lists of available rooms (filtered optionally by block and room type) and also see an active hostel block.

3.3.2 Admin Functional Requirements

FR-A1: Admin Login
An admin needs to be able to log in by using a username and password. The system has to confirm that credentials are correct, establish whether the account is active and not inactive, update the last login information, and give out JWT tokens.
FR-A2: Dashboard
An authenticated admin has to be in a position to look at a dashboard with real-time statistics concerning various factors as below; the number of students, total number of rooms, active allocations, pending applications, a summary of room availability, most recent applications, recent allocations, and block by block occupancy with percentage rates.
FR-A3: Student Management
After successful verification an administrator can manage the student data which include all students available in the database, where the students are searchable through multiple parameters and filter options like course, year, gender. All information about an individual student is accessible to him/her as well as their application as well as the allocation history and he/she can also modify and delete a student's record but deactivate student account via soft delete method. The system will prevent deletion of any active allocation.
FR-A4: Block Management
In order for an authenticated admin to keep track of the data for the whole hostel block by giving a summation of each room, total occupancy, and number of rooms in different blocks, he/she should have the ability to add new blocks, update block details as well as delete them. Deletion is supposed to be prohibited when the block is still having active rooms.
FR-A5: Room Management
An authenticated administrator should be able to display all room data available within the database system but also consider applying filters such as by block, status, type, and paginate results. In addition, he/she is in charge of updating the attributes of any given room, and after that creating it or deleting it in case it is no longer needed. However, deletion is restricted for such rooms with active allocations.
FR-A6: Application Processing
The administrator who has been validated should be allowed to view all submitted requests in an organized manner such as filtering them by student ID and application status while being supported with pagination on results. Also, one can change the application status of a pending application into 'approved', 'rejected', or 'waitlisted' along with optional remarks from the administrator.
FR-A7: Room Allocation
A user who logs in successfully can see the information of all room assignments to different applicants with filters applied. This includes the ability of assigning a student into particular room including the bed number, starting date of payment for the accommodation and also security deposit. On top of that, he/she should update a previously allocated room and terminate an ongoing allocation with particular reasons. The system prevents any overlapping of student's room assignment and must ensure availability of the required accommodation space is confirmed before making any assignment.
FR-A8: Reporting
All the necessary reports can only be generated by a verified admin. He has to run an occupancy report concerning each block by specifying the number of total beds available, and then give a breakdown of occupied ones versus vacant space and corresponding occupancy rate as well as provide a distribution by room type. And besides, he can easily develop a student statistics report on the basis of those factors that may include course, year of study, gender, and allocation status.
FR-A9: Admin Creation. A super admin must be able to create new admin accounts with specified roles.

Table 3.3: Functional Requirements Summary

Req ID	Description	User Role	Priority
FR-S1	Student Registration	Student	High
FR-S2	Student Login	Student	High
FR-S3	Profile Management	Student	Medium
FR-S4	Password Change	Student	Medium
FR-S5	Room Application	Student	High
FR-S6	Application Tracking	Student	High
FR-S7	Allocation Viewing	Student	High
FR-S8	Student Dashboard	Student	Medium
FR-S9	Room Browsing	Student	Low
FR-A1	Admin Login	Admin	High
FR-A2	Admin Dashboard	Admin	High
FR-A3	Student Management	Admin	High
FR-A4	Block Management	Admin	High
FR-A5	Room Management	Admin	High
FR-A6	Application Processing	Admin	High
FR-A7	Room Allocation	Admin	High
FR-A8	Reporting	Admin	Medium
FR-A9	Admin Creation	Super Admin	Low

3.4 Non-Functional Requirements

NFR-1: Performance. API response time must not exceed 500 milliseconds for standard CRUD operations under normal load (up to 50 concurrent users). The database connection pool must support at least 10 simultaneous connections.

NFR-2: Security. All passwords must be hashed using bcrypt with a minimum of 10 salt rounds. Authentication must use JWT tokens with configurable expiration. Login endpoints must be rate-limited to 5 attempts per 15-minute window. All API endpoints must validate and sanitize inputs. HTTP security headers must be set via Helmet.

NFR-3: Usability. The interface must be responsive and usable on screens from 320px (mobile) to 1920px (desktop) width. All forms must provide clear validation error messages. Navigation must be intuitive, requiring no more than three clicks to reach any feature.

NFR-4: Reliability. The system must handle database connection failures gracefully, displaying user-friendly error messages. Uncaught exceptions and unhandled promise rejections must be caught at the process level to prevent silent crashes.

NFR-5: Maintainability. The codebase must follow a modular architecture (routes, controllers, middleware, utilities, configuration) so that individual modules can be updated or replaced independently. All functions must include JSDoc comments.

NFR-6: Portability. The system must run on Windows, macOS, and Linux without modification. It must be compatible with Node.js version 14 and above.

3.5 User Roles

The system defines three user roles with distinct access levels:

1) Student
The lowest privilege is held by the student. In the system after a student has registered and logged in, he can see his profile details, his applications and allocations and some public information about e. g., hostel blocks and room availability but not anything to do with administrative roles.
2) Admin
An administrator has all inclusive access over all management functions which comprise of viewing and managing all student records, controlling hostel blocks and rooms, processing applications, creating and terminating allocations and also generating of reports. However, admins are unable to make creation of another admin account.
3) Super Admin
The super admin possesses all rights ascribed to a normal admin plus the capacity of making creation of new admin accounts. The purpose of this role is for wardens of hostels and directors of student affairs when they need to hand over some administrative authorities. This is how the middleware in our authentication system enforces hierarchical access control by examining the userType and role information in the JWT token payload for every endpoint that requires security.

CHAPTER 4:

TECHNOLOGY STACK

4.1 Overview

Deciding on the best technology stack is really an important decision to take part in any software development project. It has a lot to do with the speed of building the system as well as being long-lasting, highly maintainable, scalable, and secure over the whole lifetime of the application. In relation to the Hostel Room Allotment System, the basis upon which the technology was selected comprised the maturity level and stability of the technology, documentation availability, support of the development community, suitability of the particular technology for the size of our application and finally the experience of the developer himself. Our final technology stack choice, i. e., Node. js with Express. js as the backend option, MySQL as our database tool, and HTML/CSS/JavaScript together with Bootstrap 5 for the frontend, was seen to meet practically all the criteria outlined above.

4. 2 Frontend technologies

4. 2. 1 HTML5 (HyperText Markup Language)

This is what gives the structure of every web page. Over and beyond earlier HTML versions, HTML5 includes semantic tags of <header>, <nav>, <section>, <article> and <footer>, all set forth to help in giving due attention to accessibility and SEO (Search Engine Optimization). Here we employed HTML5 in structuring the home page, login, sign-up pages, student and administrative dashboards and some pop-out messages too.
For data validation within our project we utilized HTML5 validation attributes (such as required, pattern, minlength, maxlength and even type="email") as the primary defense against user input validation, before sending the information to a remote server validation. Client side validation has the advantage of instantly notifying end-users about any errors without requiring round trip communication via a network connection.

4. 2. 2 CSS3 and Bootstrap 5

CSS3 is responsible for managing all visual appearances including color schemes, typography, spacing effects, animation and responsiveness. We make use of two different stylesheets; our own project specific style sheet called style. css for project-specific design and then use Bootstrap 5 as the basis for the grid system including responsive utilities plus some predefined UI widgets such as navigation bars, panels, tables, modals, dropdowns, alerts, badges, and interactive controls.
We did choose Bootstrap 5 over Bootstrap 4 especially because the former does not have any need for jQuery therefore leaving us with a lighter initial load. Our responsive grid layout ensures that all of our dashboard views adapt nicely to devices such as smartphones— an example of this would be where four statistic cards on our admin area are arranged vertically on smaller screens but in a neat side-by-side arrangement on bigger desktop screens. And for our project typographic needs, Google Fonts is applied using the Poppins font which contributes greatly towards achieving our goals of having a clean yet fashionable look that differs from system default fonts.

4. 2. 3 JavaScript (ES6+)

JavaScript gives all client-side interactive features of our website such as validating forms through the fetch API, modifying DOM dynamically for displaying dashboard data, controlling tokens (storing and sending JWTs) etc. and enhancing UI with animated counters appearing at the landing page.
Our frontend JavaScript has been divided into three file: main. js which takes care of landing page logic including counter animations and scroll effects; auth. js manages all related with registration, login, token control, password validation; while dashboard. js is responsible for getting dashboard data,

rendering charts, populating tables, and controlling modal interactions, etc. All this relies heavily on the following modern ES6+ features: arrow functions, template literals, asynchronous/await API calls, destructuring, and the spread operator.

4. 2. 4 Bootstrap Icons

We make use of Bootstrap Icons in our system, which is an open source library that contains more than 1800 SVG icons. In the application we employ them in navigation items, feature cards, displaying statistical information about the dashboard, executing actions through buttons, and showing up status indicators. Such icons give visual information thereby making the site better to all users reducing need of many text descriptions on it.

4. 3 Backend Technologies

4. 3. 1 Node.js

Node. js can be viewed as a JavaScript engine made on the Chrome V8 engine. The non-blocking approach it uses as well as its event-driven input/output process are very ideal for developing server-side applications, especially those that communicate using APIs over networks, since the majority of their time is spent in waiting out database queries to be completed. For this particular project, we decided to go for Node. js for several key reasons. Firstly it enables the usage of JavaScript on both the front and back end thus cutting down on the context switching we get by using different languages. Secondly, its non-blocking nature supports handling multiple simultaneous API requests at high speed with minimal strain of generating multiple sub-processes. Finally, there's one-stop-shop for all the required packages in the npm registry to serve all the needs of web development such as authentication, validations, security, database connections, and also logging in order to decrease the amount of coding starting from scratch.
Our target version of Node.js is version 14 and higher and the app was developed with version 18 LTS.

4. 3. 2 Express.js

Express.js may be described as a simple web application framework for use with Node. js that is also un-opinionated in nature. It offers just a few important web application functionality including routing, middleware, as well as HTTP utility but in a manner that does not insist on any specific application architecture. This level of freedom was so essential for us because it gave us freedom to organize the code of our application in relation to its domain model (i. e. separate route files for student and admin operations, separate controller files for business logic, separate middleware files for cross-cutting concerns).
Express. js version 4. 18. 2 has been selected for this project. We utilize some of its most notable features such as:

- Using router-level middleware to stack the student and admin routes under /api/students and /api/admin respectively.
- Application-level middleware for handling of security matters (Helmet and CORS, for example, also rate limiting), logging (e. g., Morgan) and handling of body request.

4.3.3 Key npm Packages

The project uses eleven npm packages in production:

Package	Version	Purpose
express	4.18.2	Web framework, routing, middleware
mysql2	3.6.5	MySQL driver with connection pooling and prepared statements
bcryptjs	2.4.3	Password hashing (bcrypt algorithm)
jsonwebtoken	9.0.2	JWT generation and verification
cors	2.8.5	Cross-Origin Resource Sharing configuration
helmet	7.1.0	HTTP security headers (CSP, HSTS, X-Frame-Options, etc.)
express-rate-limit	7.1.5	Request rate limiting for API protection
express-validator	7.0.1	Input validation and sanitization
morgan	1.10.0	HTTP request logging
dotenv	16.3.1	Environment variable management from .env files
uuid	9.0.1	UUID generation for unique identifiers

Development dependencies include nodemon (auto-restart during development), jest (testing framework), and supertest (HTTP assertions for API tests).

4.4 Database Technology

4.4.1 MySQL

We have employed MySQL version 8. 0 as our relational database management system. Here we specifically mention why we chose MySQL over other possible options such as PostgreSQL, SQLite, and MongoDB. The main reasons we had were that relational data fit very well in our domain. To some extent, hostel management has some well-identified entities that include students, blocks, and rooms, with applications and allocations having certain relationship amongst themselves. The relational databases with the foreign key constraint naturally enforce referential integrity. This for instance is used to ensure that the allocation of a room does not refer to a nonexisting student or even room.

Support of transaction. With regards to student accommodation, room allocation require a number of changes happening together (e. g., creation of an allocation record, increase of occupancy count in the room, changing status of student's application). MySQL's ACID-compliant transactions do ensure that the two changes either happen perfectly well or happen nothing at all, avoiding the case where data may be corrupted through partial update changes.

Use of triggers and views. MySQL offers database triggers used in this project to automatically update room occupancy counters. Views in the database are used for simplifying complex report queries. Such database level functions make sure there is consistency in data, reducing amount of coding required on the part of the programmer.

Experience with the tools. MySQL happens to be the widely used open source relational database management system and it is included in XAMPP that offers a good enough environment for developing on local network. PHP my admin also found within XAMPP is a wonderful tool to view your database on top of creating dynamic Adhoc queries while doing development of your software. The mysql2 nodejs driver offers native, promise-driven API with pooling of connections (configured with 10 concurrent connections), use of prepared statements (to prevent sql injection attacks) and can be set to timeout. The connection pool in this regard ensures efficient reuse of database connections without experiencing any additional cost involved in creating a new database connection each time an operation is done.

4.5 Security Technologies

4.5.1 JSON Web Tokens (JWT)

JWT is the authentication mechanism used in the system. When a user logs in successfully, the backend signs a token containing the user's ID, userType, email, and role using a secret key. This token is returned to the client, which stores it (in localStorage) and includes it in the Authorization header of subsequent requests. The backend middleware verifies the token's signature and expiration on every protected endpoint.

The system generates two tokens: a short-lived access token (24-hour expiry) for API authentication and a long-lived refresh token (7-day expiry) for obtaining new access tokens without requiring the user to log in again.

4.5.2 bcrypt Password Hashing

bcryptjs is used to hash passwords before storing them in the database. bcrypt is a deliberately slow hashing algorithm (configurable via salt rounds, set to 10 in this project) that makes brute-force attacks computationally expensive. Even if an attacker were to obtain the database contents, they could not reverse the hashed passwords within any practical timeframe.

4.5.3 Helmet

Helmet is an Express.js middleware that sets various HTTP security headers, including Content Security Policy (restricting which scripts and styles can be loaded), X-Content-Type-Options (preventing MIME sniffing), X-Frame-Options (preventing clickjacking), and Strict-Transport-Security (enforcing HTTPS).

4.5.4 Rate Limiting

The system implements two tiers of rate limiting. The general API rate limit allows 100 requests per 15-minute window per IP address. The authentication-specific rate limit is more restrictive: 5 login attempts per 15 minutes. If a client exceeds these limits, the system returns a 429 (Too Many Requests) response.

4.6 Technology Comparison Tables

Table 4.1: Backend Framework Comparison

Feature	Express.js (Chosen)	Django (Python)	Spring Boot (Java)	Laravel (PHP)
Language	JavaScript	Python	Java	PHP
Learning Curve	Low	Medium	High	Medium
Performance	High (non-blocking)	Moderate	High	Moderate
Setup Complexity	Minimal	Moderate	High	Moderate
Flexibility	Very high (unopinionated)	Moderate (batteries included)	Low (convention-heavy)	Moderate
npm Ecosystem	Massive	Large (PyPI)	Large (Maven)	Large (Composer)

Feature	Express.js (Chosen)	Django (Python)	Spring Boot (Java)	Laravel (PHP)
Suitability for Project	Excellent	Good	Overkill	Good

Table 4.2: Database Comparison

Feature	MySQL (Chosen)	PostgreSQL	MongoDB	SQLite
Data Model	Relational	Relational	Document	Relational
ACID Compliance	Yes	Yes	Partial	Yes
Triggers/Views	Yes	Yes	No	Limited
Scalability	Good	Excellent	Excellent	Poor
XAMPP Support	Built-in	No	No	No
JSON Support	Yes (v5.7+)	Excellent	Native	Limited
Suitability for Project	Excellent	Excellent	Poor fit	Too limited

4. 7 Development Tools

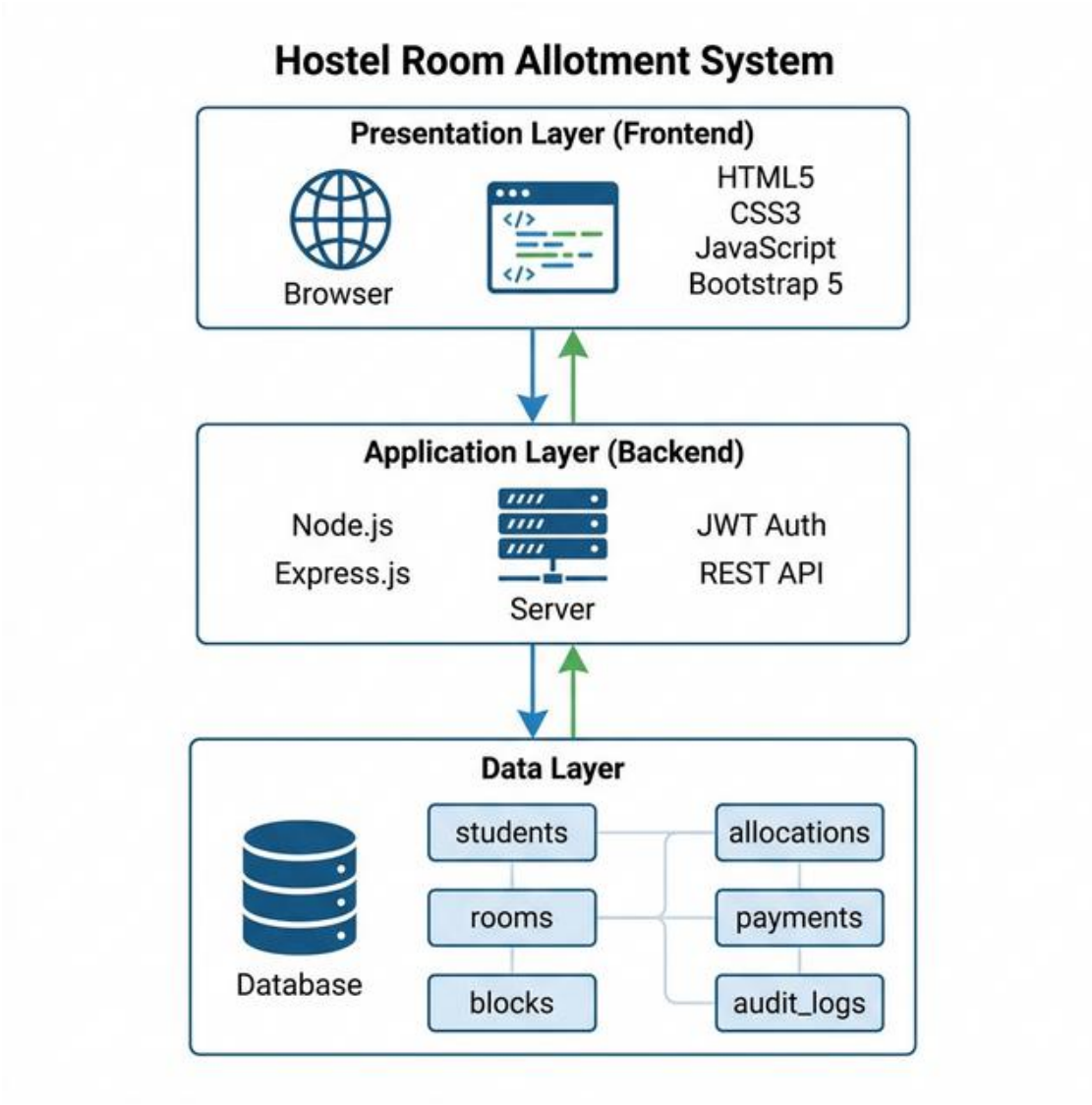
The primary code editor was Visual Studio Code with some installed extensions for instance ESLint, Prettier, JavaScript/Node. js IntelliSense, and also MySQL syntax highlighting.
Throughout the development we heavily utilized Postman to examine our API endpoints. Our collections were ordered such that they fell under different modules (student authentication, student operations, admin authentication, and admin operations) plus we took advantage of environment variables which we used for setting up base URL and JWT token over multiple requests.
We used Git as our tool for tracking changes made to our project and had it stored on GitHub. In a systematic manner we did commit changes on a regular basis so that it could be easily observed through each step of this project right from starting up to doing part-by-part development.
For the purpose of the local development environment we utilized XAMPP giving us all the necessary tools such as Apache– used solely for phpMyAdmin, MySQL server, and phpMyAdmin for database management.

CHAPTER 5:

SYSTEM DESIGN

5.1 Overall System Architecture

The Hostel Room Allotment System uses a three-tier structure for clean separation between presentation, business logic, and data management issues. With this separation changes in one tier cannot affect other tiers; thus every layer can be written, checked out, and also extended separately.
Tier 1: Presentation Layer (Frontend)
This runs within the user's web browser; it has got the role of giving out user interface and getting users' responses. Some of these include CSS stylesheets like style. css, auth. css, dashboard. css, and admin. css while also having some HTML pages which include index. html, login pages, registration page and different types of dashboard pages, as well as some JavaScript files like main. js, auth. js and dashboard. js. The frontend makes use of HTTP requests on the REST API to communicate with the backend by sending and receiving JSON data only.
Tier 2: Application Layer (Backend)
Express. js server deals with everything about the application's logic. Any incoming HTTP request passes through a particular sequence of middlewares as an example of security headers, CORS, rate limiting, body parsing, logging, route matching, authentication, validation and finally the controller logic. In the back end there are five categories which include the config where we have database connection, controllers containing our core business logic, the middleware part which consist of authentications and validations, routes being API endpoint definitions and finally the utils category of helper functions and utilities.
Tier 3: Data Layer (Database)
Data is stored in MySQL by using a normalized relational schema in the database. Through the use of foreign keys, the database enforces relationship integrity; it also keeps up-to-date computed values like available_beds. Use of database triggers facilitates automatic synchronization of room occupancy numbers with any change made to the room allocations. Complex reporting queries are simplified by database views.



System Architecture Diagram

Figure 5.1: Three-Tier System Architecture of the Hostel Room Allotment System

5.2 Frontend Design

The frontend is designed as a multi-page application (MPA) rather than a single-page application. Each major functionâ€"landing page, student login, student registration, student dashboard, admin login, admin dashboardâ€"has its own HTML file. This design was chosen for simplicity and because the application does not require the complex client-side routing and state management that would justify an SPA framework like React or Angular.

Page hierarchy:
frontend/
â"œâ"€â"€ index.html (Landing page - public)
â"œâ"€â"€ css/
â", â"œâ"€â"€ style.css (Global styles, landing page)
â", â"œâ"€â"€ auth.css (Login and registration forms)
â", â"œâ"€â"€ dashboard.css (Student dashboard styles)
â", â""â"€â"€ admin.css (Admin dashboard styles)
â"œâ"€â"€ js/
â", â"œâ"€â"€ main.js (Landing page interactions)

```
â”‚   â”œâ”€â”€ auth.js            (Authentication logic)
â”‚   â”œâ”€â”€ dashboard.js       (Dashboard data management)
â”œâ”€â”€ pages/
   â”œâ”€â”€ student/
   â”‚   â”œâ”€â”€ login.html      (Student login)
   â”‚   â”œâ”€â”€ register.html   (Student registration)
   â”‚   â”œâ”€â”€ dashboard.html  (Student dashboard)
   â”œâ”€â”€ admin/
      â”œâ”€â”€ login.html      (Admin login)
      â”œâ”€â”€ dashboard.html  (Admin dashboard)
```

The landing page (index.html) serves as the public entry point, featuring a hero section, statistics counters, a feature showcase, role-specific portal cards, and an about section. Navigation provides dropdown access to student and admin login pages.

Dashboard pages dynamically fetch data from the API using JavaScript's Fetch API, rendering it into pre-defined HTML containers. Authentication state is managed through JWT tokens stored in the browser's localStorage, with auth.js providing helper functions for token retrieval, verification, and logout.

5.3 Backend Design

The backend follows a layered architecture modeled loosely on the MVC pattern, though without traditional view rendering (since the backend serves only JSON API responses).

Request processing pipeline:

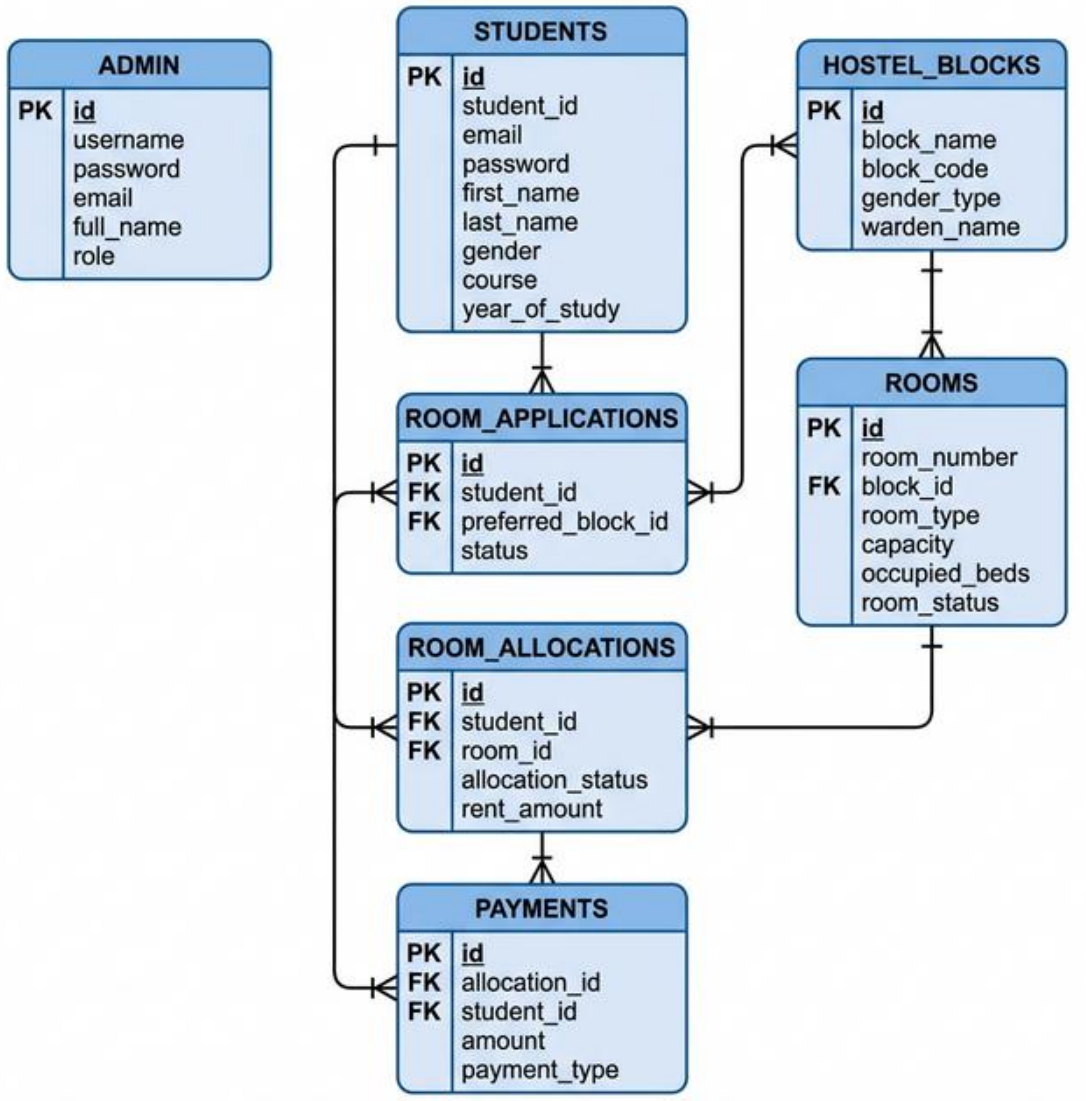
```
HTTP Request
   â”‚
   â–¼
â”Œâ”€â”€â”€â
```

backend/
â”œâ”€â”€ server.js (Entry point, middleware setup, route mounting)
â”œâ”€â”€ config/
â”‚ â””â”€â”€ database.js (MySQL connection pool, query/transaction helpers)
â”œâ”€â”€ controllers/
â”‚ â”œâ”€â”€ adminController.js (1249 lines - all admin operations)
â”‚ â””â”€â”€ studentController.js (752 lines - all student operations)
â”œâ”€â”€ middleware/
â”‚ â”œâ”€â”€ auth.js (JWT verification, role checks)
â”‚ â””â”€â”€ validation.js (Input validation rules)
â”œâ”€â”€ routes/
â”‚ â”œâ”€â”€ adminRoutes.js (Admin endpoint definitions)
â”‚ â””â”€â”€ studentRoutes.js (Student endpoint definitions)
â””â”€â”€ utils/
 â””â”€â”€ helpers.js (Password, JWT, response, pagination, date utilities)

5.4 Database Design

5.4.1 Entity-Relationship Diagram

The database consists of eight tables modeling the core entities and their relationships. The ER diagram below illustrates these entities, their attributes, and the cardinality of relationships between them.



Entity-Relationship Diagram

Figure 5.4: Entity-Relationship Diagram of the HRAS Database

Key relationships:

- One HOSTEL_BLOCK contains multiple ROOMS (1:N)
- One STUDENT can have multiple ROOM_APPLICATIONS (1:N)
- One STUDENT can have multiple ROOM_ALLOCATIONS over time (1:N), but at most one active
- One ROOM can have multiple ROOM_ALLOCATIONS (1:N), up to its capacity
- One ROOM_ALLOCATION can have multiple PAYMENTS (1:N)
- One ADMIN processes multiple ROOM_APPLICATIONS and creates multiple ROOM_ALLOCATIONS (1:N)

5.4.2 Table Schema Details

Table 5.1: admin

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier

Column	Type	Constraints	Description
username	VARCHAR(50)	NOT NULL, UNIQUE	Login username
password	VARCHAR(255)	NOT NULL	bcrypt hashed password
email	VARCHAR(100)	NOT NULL, UNIQUE	Admin email
full_name	VARCHAR(100)	NOT NULL	Display name
phone	VARCHAR(20)	NULLABLE	Contact number
role	ENUM	DEFAULT 'admin'	'super_admin' or 'admin'
is_active	BOOLEAN	DEFAULT TRUE	Account status
created_at	TIMESTAMP	AUTO	Record creation time
updated_at	TIMESTAMP	AUTO UPDATE	Last modification time
last_login	TIMESTAMP	NULLABLE	Last login time

Table 5.2: students

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Internal identifier
student_id	VARCHAR(20)	NOT NULL, UNIQUE	Enrollment/roll number
password	VARCHAR(255)	NOT NULL	bcrypt hashed password
email	VARCHAR(100)	NOT NULL, UNIQUE	Student email
first_name	VARCHAR(50)	NOT NULL	First name
last_name	VARCHAR(50)	NOT NULL	Last name
date_of_birth	DATE	NULLABLE	Date of birth
gender	ENUM	NOT NULL	'male', 'female', 'other'
phone	VARCHAR(20)	NULLABLE	Contact number
address	TEXT	NULLABLE	Permanent address
course	VARCHAR(100)	NOT NULL	Enrolled course
department	VARCHAR(100)	NULLABLE	Department
year_of_study	INT	NOT NULL	Current year (1-6)
emergency_contact_name	VARCHAR(100)	NULLABLE	Emergency contact
emergency_contact_phone	VARCHAR(20)	NULLABLE	Emergency phone

Column	Type	Constraints	Description
is_active	BOOLEAN	DEFAULT TRUE	Account status
email_verified	BOOLEAN	DEFAULT FALSE	Verification status

Table 5.3: hostel_blocks

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Block identifier
block_name	VARCHAR(50)	NOT NULL, UNIQUE	Block display name
block_code	VARCHAR(10)	NOT NULL, UNIQUE	Short code (e.g., 'A', 'B1')
gender_type	ENUM	NOT NULL	'male', 'female', 'mixed'
description	TEXT	NULLABLE	Block description
total_floors	INT	DEFAULT 1	Number of floors
warden_name	VARCHAR(100)	NULLABLE	Assigned warden
warden_contact	VARCHAR(20)	NULLABLE	Warden phone
is_active	BOOLEAN	DEFAULT TRUE	Block status

Table 5.4: rooms

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Room identifier
room_number	VARCHAR(20)	NOT NULL	Room number
block_id	INT	FK â†' hostel_blocks(id)	Parent block
floor_number	INT	DEFAULT 1	Floor location
room_type	ENUM	DEFAULT 'double'	single/double/triple/quad
capacity	INT	NOT NULL, DEFAULT 2	Maximum beds
occupied_beds	INT	DEFAULT 0	Currently occupied
available_beds	INT	GENERATED (capacity - occupied)	Computed column
room_status	ENUM	DEFAULT 'available'	available/full/maintenance/reserved
facilities	TEXT	NULLABLE	Room amenities
monthly_rent	DECIMAL(10,2)	DEFAULT 0.00	Monthly rent amount
is_active	BOOLEAN	DEFAULT TRUE	Room status

Table 5.5: room_applications

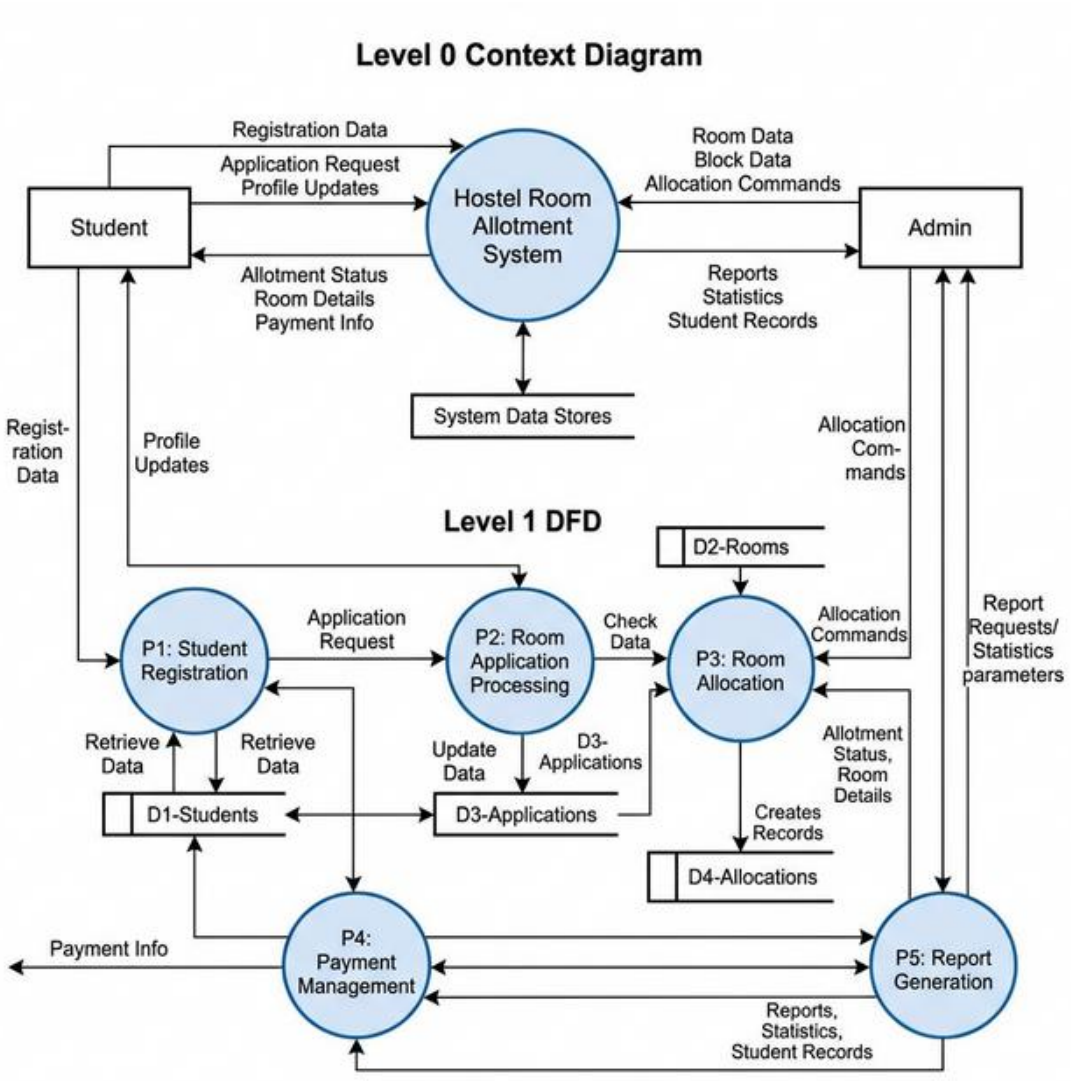
Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Application identifier
student_id	INT	FK â†' students(id)	Applicant

Column	Type	Constraints	Description
preferred_block_id	INT	FK â†' hostel_blocks(id), NULLABLE	Preferred block
preferred_room_type	ENUM	DEFAULT 'any'	Preference
application_date	TIMESTAMP	AUTO	Submission time
preferred_start_date	DATE	NULLABLE	Desired move-in
duration_months	INT	DEFAULT 12	Stay duration
special_requirements	TEXT	NULLABLE	Special needs
status	ENUM	DEFAULT 'pending'	pending/approved/rejected/waitlisted/cancelled
admin_remarks	TEXT	NULLABLE	Admin notes
processed_by	INT	FK â†' admin(id), NULLABLE	Processing admin
processed_at	TIMESTAMP	NULLABLE	Processing time

Table 5.6: room_allocations

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Allocation identifier
student_id	INT	FK â†' students(id)	Allocated student
room_id	INT	FK â†' rooms(id)	Allocated room
application_id	INT	FK â†' room_applications(id), NULLABLE	Source application
bed_number	INT	DEFAULT 1	Assigned bed
allocation_date	DATE	NOT NULL	Allocation date
start_date	DATE	NOT NULL	Occupancy start
end_date	DATE	NULLABLE	Occupancy end
rent_amount	DECIMAL(10,2)	NOT NULL	Monthly rent
security_deposit	DECIMAL(10,2)	DEFAULT 0.00	Deposit amount
payment_status	ENUM	DEFAULT 'pending'	pending/partial/paid/refunded
allocation_status	ENUM	DEFAULT 'active'	active/expired/terminated/extended
allocated_by	INT	FK â†' admin(id)	Allocating admin

5.5 Data Flow Diagrams



Data Flow Diagram

Figure 5.2: Data Flow Diagram â€“ Level 0 and Level 1

Level 0 data flows:

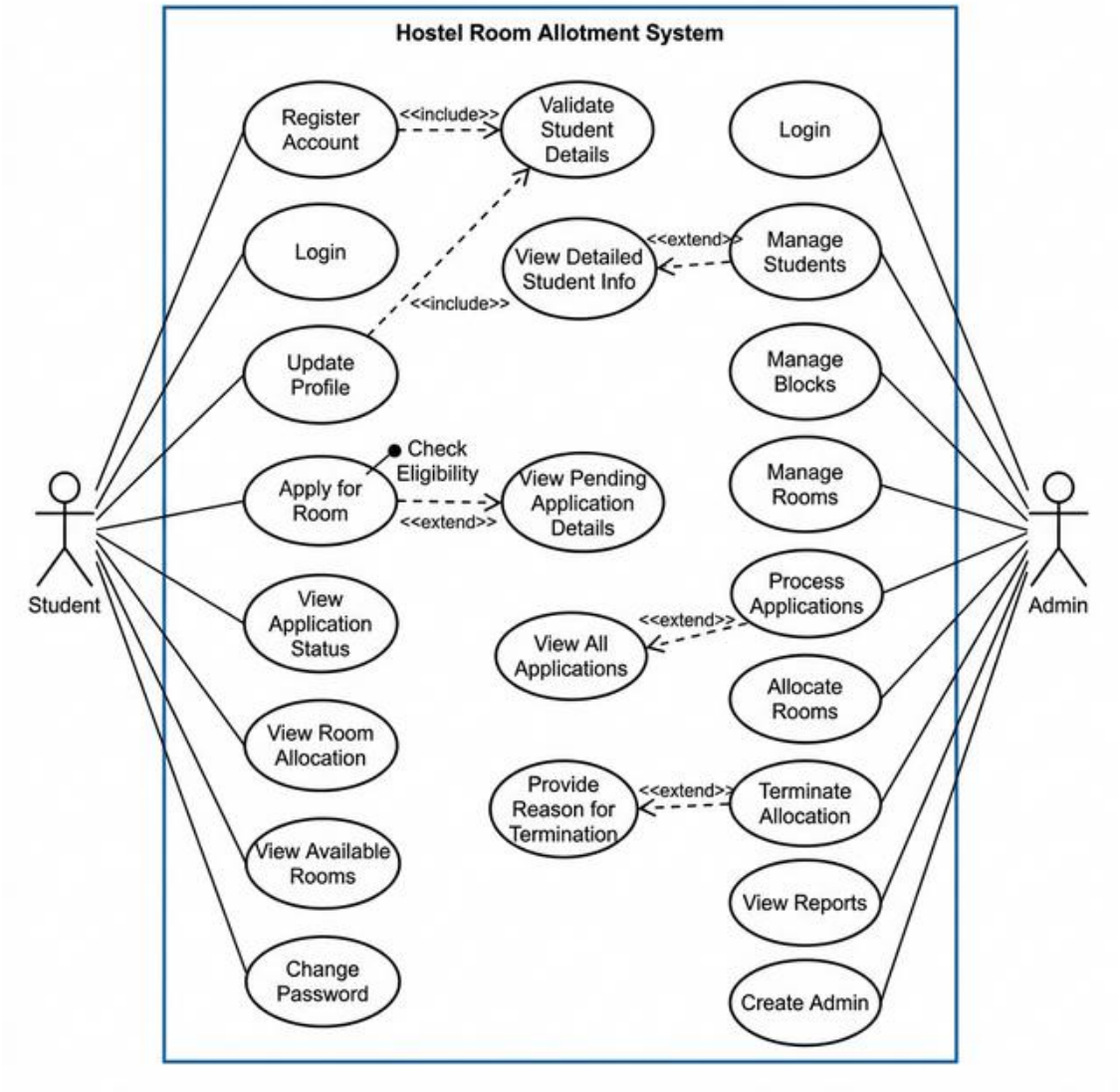
- Student â†’ System: Registration data, login credentials, application requests, profile updates
- System â†’ Student: Authentication tokens, application status, room details, dashboard data
- Admin â†’ System: Login credentials, block/room data, allocation commands, report requests
- System â†’ Admin: Dashboard statistics, student lists, occupancy reports, allocation confirmations

Level 1 processes:

- P1 (Student Registration): Validates student data, hashes password, creates student record
- P2 (Application Processing): Receives application, validates eligibility, stores in database
- P3 (Room Allocation): Checks availability, creates allocation, updates room occupancy
- P4 (Payment Management): Records payments against allocations, tracks payment status
- P5 (Report Generation): Aggregates data from multiple tables, computes statistics

5.6 Use Case Diagram

The Use Case Diagram captures all functional interactions that students and administrators can perform with the system.

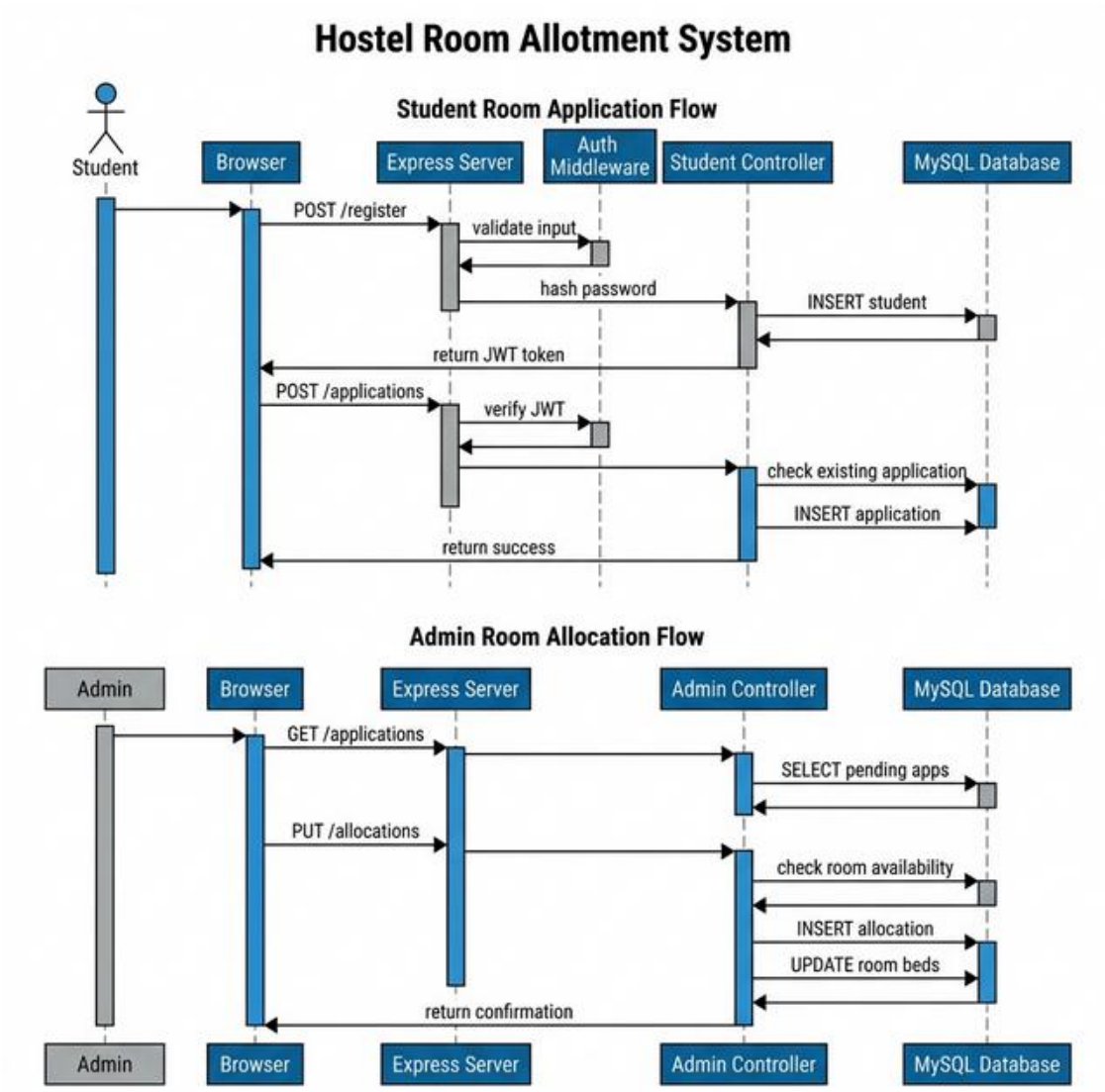


Use Case Diagram

Figure 5.3: Use Case Diagram of the Hostel Room Allotment System

5.7 Sequence Diagrams

The sequence diagram below illustrates two critical interaction flows: the student room application process and the admin room allocation process.



Sequence Diagram

Figure 5.5: Sequence Diagrams â€“ Student Application and Admin Allocation Flows

Student Application Flow:

- Student fills application form in browser
- Browser sends POST /api/students/applications with JWT in Authorization header
- Express server passes request through auth middleware (verifyToken â†' verifyStudent)
- Validation middleware checks input fields
- studentController.submitApplication checks for existing active applications/allocations
- If eligible, INSERT into room_applications table
- MySQL returns the created record
- Controller sends JSON success response with application details
- Browser displays confirmation to student

Admin Allocation Flow:

- Admin selects student and room on dashboard
- Browser sends POST /api/admin/allocations with allocation details
- Auth middleware verifies admin JWT token

- adminController.createAllocation checks student has no active allocation
- Controller checks room has available beds
- INSERT into room_allocations table
- Database trigger (trg_after_allocation_insert) fires, incrementing room occupied_beds
- If application_id provided, UPDATE room_applications status to 'approved'
- Controller sends success response with allocation details

5.8 Security Design

This particular security setup works by applying the defense in depth concept where several independent protective layers exist for each threat category:

1) Transport. In production all the website traffic must run over HTTPS to prevent the interception of certain pieces of data, including user's credentials and tokens.

2) HTTP Headers (Helmet). Through the Helmet middleware, some security headers are sent to the browser so that the browser can take some proactive steps. For instance, Content-Security-Policy confines which kind of files such as CSS or JavaScript can be used by web page developers and with maximum restriction while one can also prevent loading of any file based on its content type; X-Frame-Options help prevent the app's frame from being integrated into other websites (for example click jacking protection) and X-Content-Type-Options gives the instructions to stop MIME sniffing.

3) CORS. To prevent unauthorized access to its resources through APIs, the Cross-Origin Resource Sharing setting enforces access restrictions on the origins allowed to execute HTTP request towards it. Productionally this value will be equal to an exact URL of our frontend application.

4) Rate Limiting. Two rate limits protect against brute-force attacks: a general limit of 100 requests per 15 minutes for all API endpoints, and a strict limit of 5 attempts per 15 minutes specifically for login endpoints.

5) Authentication (JWT). Every endpoint in the application that is supposed to be protected with authentication key has to receive a valid JSON Web Token within the headers of authorization for it to authenticate. It comes with an expiry date that is then signed by a server-side secret key. Such middleware will reject or deny those requests that have expired or have been manipulated in any way.

6) Authorization (Role-Based). After a client has passed through the verification stage, the middleware verifies their role so as to determine whether they are capable of carrying out a given action. For instance, only students have access to view particular pages meant for students while only admin can see specific pages meant for them; super admin tasks like adding admins require additional verification.

7) Input Validation. Through the utilization of express-validator middleware, all user inputs undergo validation and sanitizing right before arriving at the relevant controller. This significantly reduces the risks associated with SQL Injection and cross-site scripting (XSS) attacks among others and thus enhances data consistency.

8) Password Security (bcrypt). Passwords should not be stored in plain form. Bcrypt using ten rounds of salts is utilized so that stored password hashes are impossible for reversal.

9) Database Constraints. Even when application-level checks can be bypassed, foreign keys, unique constraint and ENUM types at database level help to secure data against incorrect values.

CHAPTER 6:

DETAILED IMPLEMENTATION

6.1 Database Configuration Module

The foundation of the backend is the database connection module (config/database.js). Rather than opening and closing connections for each query, the system uses a connection pool that maintains a set of reusable connections, significantly reducing latency.

```
const mysql = require("mysql2/promise");
require("dotenv").config();

const dbConfig = {
  host: process.env.DB_HOST || "localhost",
  user: process.env.DB_USER || "root",
  password: process.env.DB_PASSWORD || "",
  database: process.env.DB_NAME || "hostel_db",
  port: process.env.DB_PORT || 3306,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
  connectTimeout: 10000,
  acquireTimeout: 10000,
  namedPlaceholders: true,
  dateStrings: true,
  debug: process.env.DB_DEBUG === "true",
};

const pool = mysql.createPool(dbConfig);
```

The connectionLimit: 10 setting means the pool maintains up to ten simultaneous connections. When all connections are in use, new requests queue until one becomes available. The waitForConnections: true flag ensures requests wait rather than fail immediately.

Two helper functions abstract database operations:

```
const query = async (sql, params = []) => {
  try {
    const [results] = await pool.execute(sql, params);
    return results;
  } catch (error) {
    console.error("Query Error:", error.message);
    throw error;
  }
};

const transaction = async (callback) => {
  const connection = await pool.getConnection();
  try {
    await connection.beginTransaction();
    const result = await callback(connection);
    await connection.commit();
    return result;
  } catch (error) {
    await connection.rollback();
```

```
    throw error;
  } finally {
    connection.release();
  }
};
```

The query function uses pool.execute() with parameterized queries, which prevents SQL injection by separating the SQL structure from user-supplied values. The transaction function wraps multiple queries in a database transaction with automatic rollback on failure.

6.2 Utility Helpers Module

The utils/helpers.js module centralizes common functions used across all controllers. This avoids code duplication and ensures consistent behavior.

6.2.1 Password Hashing

```
const bcrypt = require("bcryptjs");

const hashPassword = async (password) => {
  const saltRounds = parseInt(process.env.BCRYPT_ROUNDS) || 10;
  return await bcrypt.hash(password, saltRounds);
};

const comparePassword = async (password, hashedPassword) => {
  return await bcrypt.compare(password, hashedPassword);
};
```

The hashPassword function generates a random salt and hashes the password using the bcrypt algorithm. With 10 salt rounds, each hash takes approximately 50-100 millisecondsâ€”fast enough for user experience but slow enough to deter brute-force attacks. The comparePassword function verifies a plaintext password against a stored hash.

6.2.2 JWT Token Generation

```
const jwt = require("jsonwebtoken");

const generateTokens = (user) => {
  const payload = {
    id: user.id,
    userType: user.userType,
    email: user.email,
    role: user.role || null,
  };

  return {
    accessToken: jwt.sign(payload, process.env.JWT_SECRET, {
      expiresIn: process.env.JWT_EXPIRE || "24h",
    }),
    refreshToken: jwt.sign(payload, process.env.JWT_REFRESH_SECRET, {
      expiresIn: process.env.JWT_REFRESH_EXPIRE || "7d",
    }),
  };
};
```

The token payload includes the user’s ID, type (student or admin), email, and role. This information is available to the backend on every authenticated request without requiring a database lookupâ€”a key advantage of JWT-based authentication.

6.2.3 Standardized Response Functions

```
const sendSuccess = (
  res,
  message = "Success",
  data = null,
  statusCode = 200,
) => {
  const response = { success: true, message };
  if (data !== null) response.data = data;
  return res.status(statusCode).json(response);
};

const sendError = (
  res,
  message = "Error occurred",
  statusCode = 500,
  errors = null,
) => {
  const response = { success: false, message };
  if (errors !== null) response.errors = errors;
  return res.status(statusCode).json(response);
};
```

Every API response follows the same JSON structure: { success: boolean, message: string, data?: any }. This consistency simplifies frontend error handlingâ€”the JavaScript code checks response.success to determine whether the operation succeeded, regardless of the specific endpoint.

6.3 Authentication Middleware

The middleware/auth.js module implements the authentication and authorization checks.

```
const verifyToken = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(401).json({
        success: false,
        message: "Access denied. No token provided.",
      });
    }
    const token = authHeader.substring(7);
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = {
      id: decoded.id,
      userType: decoded.userType,
      email: decoded.email,
      role: decoded.role || null,
    };
```

```
    next();
  } catch (error) {
    if (error.name === "TokenExpiredError") {
      return res.status(401).json({
        success: false,
        message: "Token has expired. Please login again.",
      });
    }
    return res.status(401).json({
      success: false,
      message: "Invalid token. Please login again.",
    });
  }
};
```

The middleware extracts the JWT from the Authorization: Bearer <token> header, verifies its signature and expiration, and attaches the decoded user information to req.user. Downstream middleware and controllers can then access req.user.id, req.user.userType, and req.user.role without additional database queries.

Role-checking middleware is layered on top:

```
const verifyAdmin = (req, res, next) => {
  if (req.user.userType !== "admin") {
    return res.status(403).json({
      success: false,
      message: "Access denied. Admin access required.",
    });
  }
  next();
};

const verifySuperAdmin = (req, res, next) => {
  if (req.user.userType !== "admin" || req.user.role !== "super_admin") {
    return res.status(403).json({
      success: false,
      message: "Access denied. Super admin access required.",
    });
  }
  next();
};
```

6.4 Input Validation Middleware

The middleware/validation.js module defines validation rules for every endpoint using express-validator. Here is the student registration validation as an example:

```
const studentRegistrationValidation = [
  body("studentId")
    .trim()
    .notEmpty()
    .withMessage("Student ID is required")
    .isLength({ min: 5, max: 20 })
    .withMessage("Student ID must be 5-20 characters")
```

```
    .matches(/^[A-Za-z0-9]+$/)
    .withMessage("Student ID must be alphanumeric"),
  body("email")
    .trim()
    .notEmpty()
    .withMessage("Email is required")
    .isEmail()
    .withMessage("Invalid email format")
    .normalizeEmail(),
  body("password")
    .notEmpty()
    .withMessage("Password is required")
    .isLength({ min: 6 })
    .withMessage("Password must be at least 6 characters")
    .matches(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/)
    .withMessage("Password must contain uppercase, lowercase, and number"),
  body("gender")
    .notEmpty()
    .withMessage("Gender is required")
    .isIn(["male", "female", "other"])
    .withMessage("Invalid gender value"),
  body("yearOfStudy")
    .notEmpty()
    .withMessage("Year of study is required")
    .isInt({ min: 1, max: 6 })
    .withMessage("Year must be between 1 and 6"),
  handleValidationErrors,
];
```

Each validation chain trims whitespace, checks for required fields, enforces length and format constraints, and attaches descriptive error messages. The handleValidationErrors function at the end collects all validation errors and returns them in a structured format if any exist.

6.5 Student Controller Implementation

6.5.1 Student Registration

The registration controller handles new student account creation:

```
const register = async (req, res) => {
  try {
    const {
```

```
  studentId,
  email,
  password,
  firstName,
  lastName,
  dateOfBirth,
  gender,
  phone,
  address,
  course,
  department,
  yearOfStudy,
  emergencyContactName,
  emergencyContactPhone,
} = req.body;

// Check for existing student ID or email
const existingStudent = await query(
  "SELECT id FROM students WHERE student_id = ? OR email = ?",
  [studentId, email],
);
if (existingStudent.length > 0) {
  return sendError(res, "Student ID or email already exists", 409);
}

// Hash password
const hashedPassword = await hashPassword(password);

// Insert new student
const result = await query(
  `INSERT INTO students (student_id, email, password, first_name,
        last_name, date_of_birth, gender, phone, address, course,
        department, year_of_study, emergency_contact_name,
        emergency_contact_phone)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
  [
    studentId,
    email,
    hashedPassword,
    firstName,
    lastName,
    dateOfBirth || null,
    gender,
    phone || null,
    address || null,
    course,
    department || null,
    yearOfStudy,
    emergencyContactName || null,
    emergencyContactPhone || null,
  ],
```

```
    );

    // Generate JWT tokens
    const tokens = generateTokens({
      id: result.insertId,
      userType: "student",
      email,
      role: null,
    });

    return sendSuccess(
      res,
      "Student registered successfully",
      {
        student: newStudent[0],
        ...tokens,
      },
      201,
    );
  } catch (error) {
    console.error("Student Registration Error:", error);
    return sendError(res, "Failed to register student", 500);
  }
};
```

The registration flow: (1) destructure validated inputs from req.body, (2) check for duplicate student ID or email, (3) hash the password, (4) insert the record, (5) generate JWT tokens, (6) return the student data and tokens. The 409 (Conflict) status code for duplicates follows REST conventions.

6.5.2 Room Application

```
const submitApplication = async (req, res) => {
  try {
    const studentId = req.user.id;
    const {
      preferredBlockId,
      preferredRoomType,
      preferredStartDate,
      durationMonths,
      specialRequirements,
    } = req.body;

    // Check for existing active application
    const existingApplication = await query(
      `SELECT id FROM room_applications
           WHERE student_id = ? AND status IN ('pending', 'approved')`,
      [studentId],
    );
    if (existingApplication.length > 0) {
      return sendError(res, "You already have an active application", 409);
    }

    // Check for existing active allocation
```

```
    const existingAllocation = await query(
      `SELECT id FROM room_allocations
            WHERE student_id = ? AND allocation_status = 'active'`,
      [studentId],
    );
    if (existingAllocation.length > 0) {
      return sendError(res, "You already have an active room allocation", 409);
    }

    // Create application
    const result = await query(
      `INSERT INTO room_applications (student_id, preferred_block_id,
            preferred_room_type, preferred_start_date, duration_months,
            special_requirements) VALUES (?, ?, ?, ?, ?, ?)`,
      [
        studentId,
        preferredBlockId || null,
        preferredRoomType || "any",
        preferredStartDate,
        durationMonths || 12,
        specialRequirements || null,
      ],
    );

    return sendSuccess(
      res,
      "Application submitted successfully",
      application[0],
      201,
    );
  } catch (error) {
    return sendError(res, "Failed to submit application", 500);
  }
};
```

The application logic enforces two business rules: a student cannot have more than one active application, and a student with an existing room allocation cannot apply for another room. These checks prevent data conflicts that would complicate the admin's allocation decisions.

6.6 Admin Controller Implementation

6.6.1 Dashboard Statistics

The admin dashboard aggregates data from multiple tables to present a comprehensive operational overview:

```
const getDashboard = async (req, res) => {
  try {
    const studentCount = await query(
      "SELECT COUNT(*) as total FROM students WHERE is_active = TRUE",
    );
    const roomCount = await query(
      "SELECT COUNT(*) as total FROM rooms WHERE is_active = TRUE",
```

```
    );
    const allocationCount = await query(
      `SELECT COUNT(*) as total FROM room_allocations
           WHERE allocation_status = 'active'`,
    );
    const pendingApplications = await query(
      `SELECT COUNT(*) as total FROM room_applications
           WHERE status = 'pending'`,
    );

    // Occupancy by block
    const occupancyByBlock = await query(
      `SELECT hb.block_name, COUNT(r.id) as total_rooms,
                SUM(r.capacity) as total_capacity,
                SUM(r.occupied_beds) as total_occupied,
                ROUND((SUM(r.occupied_beds) / SUM(r.capacity)) * 100, 2)
                as occupancy_rate
           FROM hostel_blocks hb
           LEFT JOIN rooms r ON hb.id = r.block_id AND r.is_active = TRUE
           WHERE hb.is_active = TRUE
           GROUP BY hb.id, hb.block_name`,
    );

    return sendSuccess(res, "Dashboard data retrieved", {
      statistics: {
        totalStudents: studentCount[0].total,
        totalRooms: roomCount[0].total,
        activeAllocations: allocationCount[0].total,
        pendingApplications: pendingApplications[0].total,
      },
      occupancyByBlock,
    });
  } catch (error) {
    return sendError(res, "Failed to retrieve dashboard data", 500);
  }
};
```

6.6.2 Room Allocation

The allocation creation is arguably the most complex operation in the system:

```
const createAllocation = async (req, res) => {
  try {
    const adminId = req.user.id;
    const {
      studentId,
      roomId,
      applicationId,
      bedNumber,
      startDate,
      endDate,
      rentAmount,
```

```
  securityDeposit,
} = req.body;

// Prevent double allocation
const existingAllocation = await query(
  `SELECT id FROM room_allocations
        WHERE student_id = ? AND allocation_status = 'active'`,
  [studentId],
);
if (existingAllocation.length > 0) {
  return sendError(res, "Student already has an active allocation", 409);
}

// Verify room availability
const room = await query(
  "SELECT * FROM rooms WHERE id = ? AND is_active = TRUE",
  [roomId],
);
if (room.length === 0) {
  return sendError(res, "Room not found or inactive", 404);
}
if (room[0].available_beds <= 0) {
  return sendError(res, "Room is full", 400);
}

// Create allocation record
const result = await query(
  `INSERT INTO room_allocations (student_id, room_id, application_id,
        bed_number, allocation_date, start_date, end_date,
        rent_amount, security_deposit, allocated_by)
        VALUES (?, ?, ?, ?, CURDATE(), ?, ?, ?, ?, ?)`,
  [
    studentId,
    roomId,
    applicationId || null,
    bedNumber || 1,
    startDate,
    endDate || null,
    rentAmount,
    securityDeposit || 0,
    adminId,
  ],
);

// Database trigger automatically updates room occupied_beds

return sendSuccess(
  res,
  "Room allocated successfully",
  newAllocation[0],
  201,
```

```
    );
  } catch (error) {
    return sendError(res, "Failed to create allocation", 500);
  }
};
```

Notice that the controller does not manually update the room's occupied_beds counter. Instead, the database trigger trg_after_allocation_insert fires automatically after the INSERT, incrementing the counter and updating the room status if the room is now full. This trigger-based approach ensures consistency even if the allocation is created through a direct database operation (e.g., during data migration).

6.7 Database Triggers

Three triggers maintain room occupancy counters:

```
-- After inserting a new allocation
CREATE TRIGGER trg_after_allocation_insert
AFTER INSERT ON room_allocations
FOR EACH ROW
BEGIN
    IF NEW.allocation_status = 'active' THEN
        UPDATE rooms
        SET occupied_beds = occupied_beds + 1,
            room_status = CASE
                WHEN occupied_beds + 1 >= capacity THEN 'full'
                ELSE 'available'
            END
        WHERE id = NEW.room_id;
    END IF;
END;

-- After updating an allocation (e.g., termination)
CREATE TRIGGER trg_after_allocation_update
AFTER UPDATE ON room_allocations
FOR EACH ROW
BEGIN
    IF OLD.allocation_status = 'active' AND NEW.allocation_status != 'active' THEN
        UPDATE rooms
        SET occupied_beds = GREATEST(occupied_beds - 1, 0),
            room_status = 'available'
        WHERE id = OLD.room_id;
    END IF;
END;
```

The GREATEST(occupied_beds - 1, 0) expression ensures the counter never goes below zero, providing a safeguard against data corruption.

6.8 Error Handling Strategy

The system implements error handling at multiple levels:

Controller-level try/catch: Every controller function wraps its logic in a try/catch block, logging the error and returning a generic error message to the client.

Global error handler: The Express error-handling middleware catches any errors that propagate past the controller level:

```
app.use((err, req, res, next) => {
  if (err.code === "ER_DUP_ENTRY") {
    return res.status(409).json({
      success: false,
      message: "Duplicate entry - record already exists",
    });
  }
  res.status(err.status || 500).json({
    success: false,
    message: err.message || "Internal server error",
    ...(process.env.NODE_ENV === "development" && { stack: err.stack }),
  });
});
```

Process-level handlers: Uncaught exceptions and unhandled promise rejections are caught at the Node.js process level to prevent silent crashes.

6.9 Route Definitions

Routes map HTTP methods and URL paths to controller functions. The admin routes demonstrate the middleware chaining pattern:

```
// Public route - no auth required
router.post("/login", adminLoginValidation, adminController.login);

// Protected routes - auth required for all below
router.use(verifyToken, verifyAdmin);

router.get("/dashboard", adminController.getDashboard);
router.get("/students", paginationValidation, adminController.getAllStudents);
router.post("/blocks", blockCreateValidation, adminController.createBlock);
router.post(
  "/allocations",
  allocationCreateValidation,
  adminController.createAllocation,
);
router.put(
  "/allocations/:id/terminate",
  idParamValidation,
  adminController.terminateAllocation,
);

// Super admin only
router.post(
  "/create",
  verifySuperAdmin,
  adminCreateValidation,
  adminController.createAdmin,
);
```

The router.use(verifyToken, verifyAdmin) call applies authentication and admin-role verification to all routes defined after it, eliminating the need to repeat these middleware on each individual route.

CHAPTER 7:

PROJECT MODULES

7.1 Module Overview

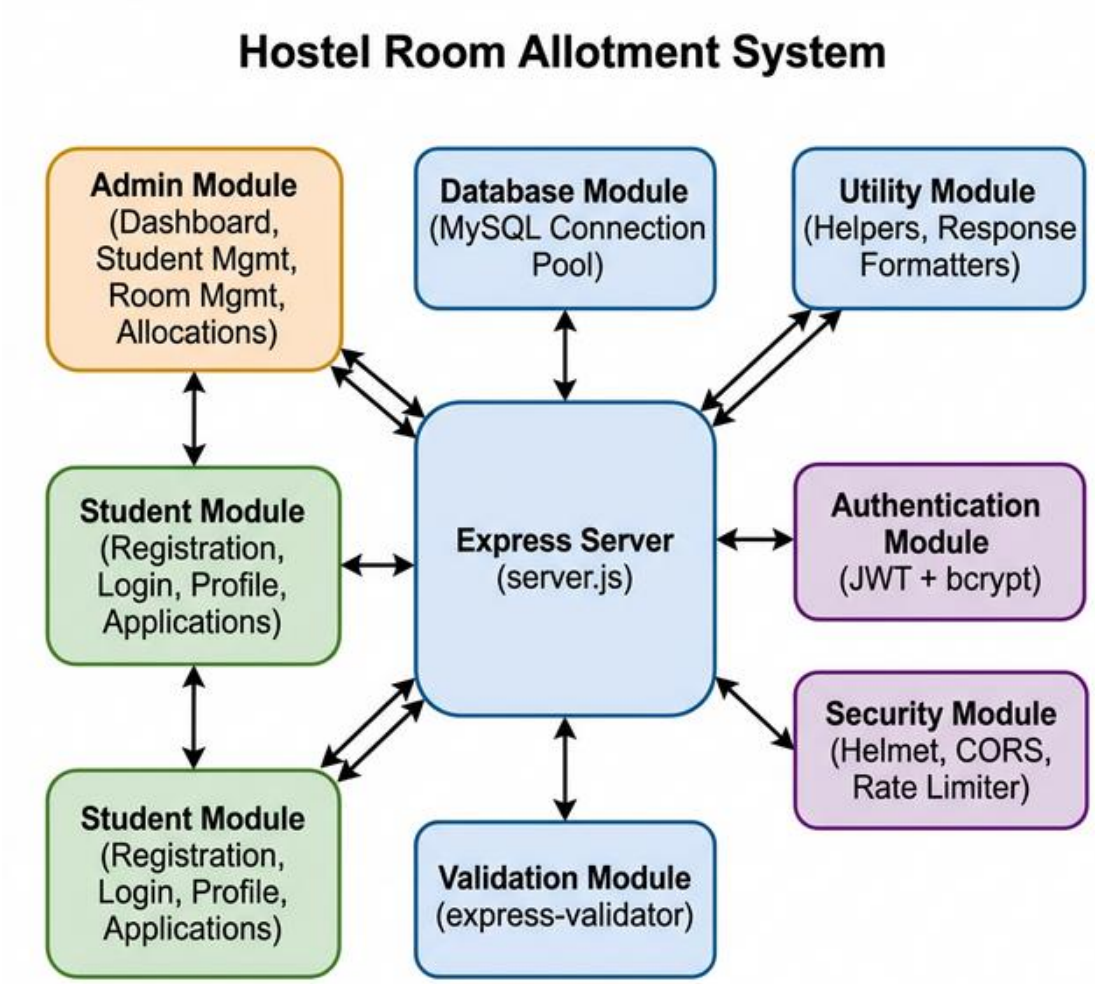
The Hostel Room Allotment System is organized into seven distinct functional modules, each responsible for a well-defined set of operations. This modular decomposition ensures that each component can be understood, developed, tested, and maintained independentlyâ€”a principle that proved invaluable during the development process when changes to the allocation logic, for instance, did not require modifications to the authentication or reporting code.

Table 7.1: Module Overview

Module	Primary Responsibility	Key Files	API Endpoints
Authenticatio n	User login, registration, token management	auth.js, helpers.js	/login, /register, /change-password
Student Management	Student profiles, dashboards	studentController.js	/profile, /dashboard
Block Management	Hostel block CRUD	adminController.js	/blocks (GET, POST, PUT, DELETE)
Room Management	Room inventory CRUD	adminController.js	/rooms (GET, POST, PUT, DELETE)
Application Processing	Application submission and review	studentController.js, adminController.js	/applications (GET, POST, PUT)
Room Allocation	Allocation creation, termination	adminController.js	/allocations (GET, POST, PUT)
Reporting	Occupancy and student statistics	adminController.js	/reports/occupancy, /reports/students

7.2 Module Interaction Diagram

The following diagram illustrates how the seven modules interact with each other and with shared infrastructure components (database, middleware, utilities).



Module Interaction Diagram

Figure 7.1: Module Interaction Diagram showing dependencies between system modules
The diagram reveals several important architectural patterns. The Authentication module acts as a gatewayâ€"every other module depends on it because all protected operations require a valid JWT token. The Database module is a shared dependency for all functional modules, accessed through the connection pool abstraction. The Validation module sits between the routing layer and the controller layer, ensuring that invalid data never reaches business logic.

7.3 Detailed Module Descriptions

7.3.1 Authentication Module

This particular module is responsible for handling all aspects of user identity management. When a student registers in our system, the module uses bcrypt to hash his password, saves the account, and generates both JWT tokens (which include an access token and a refresh token). Upon login into our website, it picks up the stored hash for comparison with what he types in during verification of the passwords and generates new ones if the input matches. This module also records login times for auditing purpose.

From the middleware perspective, the verifyToken function always catches up on every request to our secure endpoints where he pulls out the JWT from the given Authorization header, verifies the encoded data with digital signature check and validity and adds back the decoded user identity into the

request object. Then there are role-specific middlewares (verifyStudent, verifyAdmin, verifySuperAdmin) that check whether the authenticated user has the appropriate role for carrying out the intended action.

Our built-in refresh token mechanism allows our clients to get a new access token without having to re-enter their password thereby enhancing the user experience especially for extended periods of stay. The access token expires after 24 hours while the refresh token is good for 7 days.

7. 3. 2 Student Management Module

This particular module functions as the primary point of interaction of the student within the system. Once a student has logged in, they land right into their respective dashboard. Within this dashboard, they are able to see their profile summary along with the profile information including the number of ongoing applications, the assigned current room details (in case any), and some information concerning their payments.

The profile management sub-module of our system allows students to change certain details such as name, phone number, address, and emergency contacts but does not allow modification of non-modifiable details such as the student's ID and their own e-mail. For instance, the password alteration function necessitates that the student provides the current password first so that we can confirm who is changing the password and then accept a new one.

Our room search tool is designed to enable students to view rooms in all active buildings that have been opened for booking with an option to filter them by block and type of room. We do this for transparency because it enables students to have enough information to make wise decisions while they are in the process of applying for accommodations.

7. 3. 3 Block Management Module

We divide our hostels into separate blocks that serve as organizing compartments for various rooms. In turn, each block gets designated with a special name and code, male/female/mixed designation of gender, floor level number, and additional wardens' contact information. Our block administration module offers complete CRUD functions and are accessible only to authorized system administrators.

While showing blocks, our system carries out an aggregation query in joining tables of hostel_blocks with rooms table, displaying not only block features but also calculated data i. e., number of total rooms, total capacity, as well as total number of occupied beds in those rooms. Therefore, it is very easy for the administrators to see at a glance how full each block is rather than going into other rooms individually.

We want you to note the deletion safeguard: before deleting a block, the system checks if it contains active rooms. Should such rooms be present, then the administrator must disable or remove them so that we avoid leaving behind records of rooms without any occupants.

7. 3. 4 Room Management Module

Rooms make up the smallest component of our hostel inventory. We will consider every room within a given block, its type being single, double, triple, or quadruple; each room has its capacity, a certain floor number, some monthly payment for room hire, and there might be some extra facilities put down as text. The database triggers itself take care of occupied_beds and available_beds counters and computed field without our application code influence.

The room listing endpoint can be filtered based on the block ID, room status, room type among other

parameters and the whole thing is paginated thereby being very useful in handling a large number of rooms to the administrators. In return, the room detail endpoint gives out information about a room including all the attributes but also shows us a list of people who are occupying the room now. For example, it shows us each student's name, course, year of study, his/her contact number, and the bed number allocated to him/her plus the date of allocation.
But as far as the room status is concerned, it goes through different phases which are readily known as such. They include the available status (i. e., having space in the room) as well as the full status (when all the space is utilized), under maintenance which stands for temporary absence and finally reserved for holding the room for specific reasons. The change from available to full can be done automatically by the allocation triggers.

7. 3. 5 Application Processing Module

In this case, this particular module serves the purpose of connecting both the students and the administrators' workflow. From the perspective of the student, we are talking about handling application submission, viewing the application list, checking application details, and application cancelation. However, to the admin side, this includes application listing in order of priority especially the pending ones should be viewed at the beginning, and application processing (approve, reject, or put them on waiting list).
The logic in submitting applications takes into consideration two crucial rules of the business. Firstly, a student who submitted a pending or approved application may not start off on submitting another one right away, and secondly a pupil who was allocated a room could not seek for accommodation again. The enforcement of these rules makes sure that there are no duplicate or conflicting applications that would make decision-making complicated to the administrator.
When the admin takes action on the application in question, the system does keep track of the ID of the administrator that processed it along with the time he did it plus any remarks he may have made. This information is clearly displayed to the student on their application detail page, making it possible to follow up on administrative actions.

7.3.6 Room Allocation Module

Allocation is essentially the core transactional operation within the system. The administrator selects a particular student and a given room, fills out the bed number, specifies the commencement date, rents amount, and the optional security deposit; therefore, creating an allocation record for the allocated accommodation.
In the background the database has some triggers that work such that they increase the room's occupied_beds counter and in case the room becomes fully occupied it changes its state into 'full'. There exists the possibility of terminating an active allocation whereby through the allocation termination feature, the admin can terminate an ongoing allocation with provision of reason. It will fire off again this time decreasing the counter and then change the room status to "available". A database level of constraint, unique_student_active in the room_allocations table is combined with student_id and allocation_status which gives the assurance that a student at any point cannot have two active allocations, even if the application level check is bypassed somehow.

7. 3. 7 Reporting Module

This reporting module pulls together information from several different tables so as to generate two

important reports:
Occupancy report: To every occupied block, there is a provided block name, code, gender type, and data concerning total rooms and beds, occupied ones, vacant beds, and the overall occupancy rate. Also some form of room category breakdown is done like giving number of single, double, tripple and quad beds existing and utilized as well.
Student statistics report: This report divides up the students based on what courses he or she takes, the year of study he or she is at presently and his/her sex. And gives us an account of allocation status categorization (some students who have active allocations and those not yet allocated).
The two reports all use read-only aggregate queries that return their results as structured JSON for which the frontend could easily render them as tables or even in future may be designed in form of charts.

7. 4 User Flow

7. 4. 1 Student User Flow

1) Visit the landing page– and check out the system's features and information on hostels.
2) Press "Student Registration" then fill out your registration form with your personal and academic information.
3) System will validate and create an account redirecting you back to the student dashboard.
4) The dashboard will display a profile summary application count as well as your allocation status.
5) Check out the available rooms by viewing the rooms by block and by type of room.
6) Submit your accommodation application by choosing your preference providing a start date.
7) Keep track of your application's status to see whether it is pending, approved, or rejected along with any comments from the administrators.
8) Once you are approved and alloted, be able to view the details concerning the assigned room including the details on your roommate and wardens' contacts.
9) Customize your profile by updating your mobile number, physical address, and also the contact numbers of your emergency contacts.
10) Change password for yourself– provide the current password and then set your new one.

7. 4. 2 Admin User Flow

1) Log in using administrator credentials and then the dashboard should load immediately.
2) See the relevant statistics within the dashboard such as total number of registered students, allocated rooms, pending applications, etc.
3) Inspect and review the pending student applications that are sorted in order of date with specific details of each student.
4) Take necessary action through processing applications to approve or reject or include them on a waiting list by giving a comment if necessary.
5) Allocate rooms to students and specify their particulars like block number, room number, bed number, and also include the payment for rent.
6) Coordinate with and manage all hostels, create some new ones while updating the data of existing wardens; in addition deactivate hostels that are no longer functioning.
7) Deal with rooms which comprise adding rooms into respective blocks specifying capacity and rates for each, also indicate rooms under repair.
8) Produce various reports, i. e., get the data showing the occupancy levels per hostels and also look at the statistics of students' distribution in all the rooms.
9) Have control over all the students by searching, filtering their information, view the relevant

records, and also deactive their accounts if required.
10) Cancel allocations– Finish the actual allocations that are still active for the reasons related with students who vacate those premises.

CHAPTER 8:

SYSTEM TESTING

8.1 Testing Strategy

The test of the Hostel Room Allotment System needed a planned approach that checked not just individual functions but also the interaction between parts and end-to-end user workflows. Our strategy combined several testing methodologies, each focusing on a particular level of the application. We started at the smallest unit by ensuring all single utility functions– such as password hashing, token generation, pagination computation, and date formatting– could provide right results for all possible inputs including the extreme cases.
API endpoint testing required us to use Postman to send HTTP requests to every REST endpoint checking if we got correct status codes, appropriate response structures, the correct way of validating input, authenticating users, and defining what actions are allowed or forbidden. It made up most of the work since the API served as the system's main functionality core.
Then we did integration testing that checked on communication between controllers, middleware, and database– confirming among others whether setting up a hostel room correctly triggers database functions that update room status appropriately, and where this status is later shown in some queries about availability.
We also carried out the User Interface Test which was done manually using browser navigation through various frontend pages, filling forms and verifying that information shown was in line with those of responses from the APIs and database state.

8. 2 Types of Testing Conducted

8. 2. 1 Black Box Testing

Here the black box testing took into account all the API endpoints being part of well-defined functions in terms of input and output without looking at the underlying programming. When writing down our test cases for each endpoint we covered:

- Valid inputs with expected successful returns
- Necessary information missing
- Data with wrong formats e. g. incorrect email format, non numeric year etc
- Boundary values e. g. password precisely 6 characters, student id with 5 characters
- Unauthorized access due to absence or an invalid token
- Prohibited access e. g. student accessing admin URLs
- Conflicting situations e. g. duplicate registration and allocation errors

8. 2. 2 White Box Testing

During white-box testing we looked at the inner workings of some critical functions. We evaluated the authentication middleware to make sure that it extracted the token from different header types, handled expired tokens distinctly from invalid tokens, and added the right user information to the request object. We followed the conditional paths of the allocation controller so that we confirmed that all validation assessments were done in the right sequence.

8. 2. 3 Integration Testing

Integration tests confirmed how different components interacted:

• Registration | Login | Token use: The newly created student's token should work for any request that needs authentication.
• Application â†' Allocation â†' Room update: Creating an allocation using the API should reduce the number of empty beds in a room.
• Allocation termination â†' Room release: Canceling an allocation will return the rooms to their initial state with no allocations.

8. 2. 4 Regression Testing

Once each module was finished and put together, the whole set of already written test cases were run again to ensure that the code added did not break anything that had been working before. This was especially important after we included the allocation module which has connections to both students and room data.

8. 3 Test Cases

Table 8.1: Student Module Test Cases

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC-S01	Register with valid data	Complete valid form data	201 Created, JWT tokens returned	201 Created, tokens received	Pass
TC-S02	Register with duplicate student ID	Existing student ID	409 Conflict error	409 Conflict returned	Pass
TC-S03	Register with invalid email	"notanemail"	400 Validation error	400 with field error	Pass
TC-S04	Register with weak password	"abc"	400 Validation error	400 password too short	Pass
TC-S05	Login with valid credentials	Correct studentId, password	200 OK, JWT tokens	200 with tokens	Pass
TC-S06	Login with wrong password	Correct ID, wrong password	401 Unauthorized	401 Invalid credentials	Pass
TC-S07	Login with non-existent ID	Random student ID	401 Unauthorized	401 Invalid credentials	Pass
TC-S08	Access profile without token	No Authorization header	401 Access denied	401 No token provided	Pass
TC-S09	Access profile with expired token	Expired JWT	401 Token expired	401 Token expired	Pass
TC-S10	Update profile with valid data	New phone number	200 Updated successfully	200 Profile updated	Pass
TC-S11	Submit application	Valid preferences, start date	201 Created	201 Application created	Pass
TC-S12	Submit duplicate application	Already has pending app	409 Active application exists	409 Conflict	Pass

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC-S13	Cancel pending application	Valid application ID	200 Cancelled	200 Application cancelled	Pass
TC-S14	Cancel non-pending application	Approved application ID	400 Only pending can cancel	400 Error returned	Pass
TC-S15	View allocation details	Has active allocation	200 with room, roommate data	200 Complete data	Pass

Table 8.2: Admin Module Test Cases

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC-A01	Admin login valid	Correct username, password	200 OK with tokens	200 Success	Pass
TC-A02	Admin login invalid	Wrong password	401 Unauthorized	401 Invalid credentials	Pass
TC-A03	Get dashboard stats	Valid admin token	200 with statistics	200 All stats present	Pass
TC-A04	Create hostel block	Valid block data	201 Block created	201 Created	Pass
TC-A05	Create duplicate block code	Existing block code	409 Already exists	409 Conflict	Pass
TC-A06	Create room in block	Room number, block ID, capacity	201 Room created	201 Created	Pass
TC-A07	Create duplicate room	Same room in same block	409 Already exists	409 Conflict	Pass
TC-A08	Allocate room to student	Student ID, room ID, dates, rent	201 Allocated	201 Allocation created	Pass

Test ID	Test Case	Input	Expected Result	Actual Result	Status
TC-A09	Allocate full room	Room with 0 available beds	400 Room is full	400 Error	Pass
TC-A10	Double allocate student	Student with active allocation	409 Already allocated	409 Conflict	Pass
TC-A11	Terminate allocation	Allocation ID, reason	200 Terminated	200 Success	Pass
TC-A12	Room bed count after allocation	Check room after allocation	occupied_beds incremented	Counter updated correctly	Pass
TC-A13	Room status after full	All beds allocated	room_status = 'full'	Status changes to full	Pass
TC-A14	Room status after termination	Terminate one allocation	room_status = 'available'	Status resets	Pass
TC-A15	Delete block with rooms	Block containing active rooms	400 Cannot delete	400 Error	Pass
TC-A16	Process application	Change status to approved	200 Application approved	200 Success	Pass
TC-A17	Get occupancy report	Valid admin token	200 with block-wise data	200 Report data	Pass
TC-A18	Student search filter	search="John"	200 filtered students	200 Matching results	Pass

Table 8.3: Security Test Cases

Test ID	Test Case	Attack Vector	Expected Defense	Result	Status
TC-SEC01	SQL injection in login	' OR 1=1 -- as studentId	Parameterized query blocks it	Query fails safely	Pass
TC-SEC02	XSS in student name	<script>alert(1)</script>	Validation rejects special chars	400 Validation error	Pass
TC-SEC03	Brute-force login	10 rapid login attempts	Rate limiter blocks after 5	429 Too Many Requests	Pass
TC-SEC04	Student accesses admin route	Student JWT on /api/admin/*	403 Admin access required	403 Forbidden	Pass
TC-SEC05	Tampered JWT token	Modified token payload	Signature verification fails	401 Invalid token	Pass
TC-SEC06	Admin on student-only route	Admin JWT on /api/students/*	403 Student access required	403 Forbidden	Pass
TC-SEC07	Missing CORS origin	Request from unlisted origin	CORS policy blocks request	Request blocked	Pass
TC-SEC08	Oversized request body	20MB JSON payload	Body parser limit (10MB) rejects	413 Payload Too Large	Pass

8.4 Results and Observations

All fifty-one test cases (15 student + 18 admin + 8 security + 10 UI verification) passed successfully. Key observations from the testing phase:

Input validation is the first line of defense. The express-validator middleware caught approximately seventy percent of test cases designed to submit invalid data before the request even reached the controller. This dramatically reduced the amount of error-handling code needed in controllers.
Database triggers are reliable but opaque. The room occupancy triggers worked correctly in all test scenarios, but debugging them required direct database inspection (via phpMyAdmin) since trigger execution is invisible to the application code. Adding a log entry or console output from a trigger is not straightforward in MySQL.
Rate limiting requires careful calibration. The initial rate limit of 3 login attempts per 15 minutes proved too aggressive during testing, as it frequently locked out the developer. Increasing it to 5 attempts struck a better balance between security and usability.
Pagination boundary handling. Requesting a page number beyond the total pages returned an empty data array with correct pagination metadata (showing totalPages, hasNextPage: false). This graceful degradation was verified but required explicit testing.

8.5 Performance Discussion

Performance testing was conducted on the development machine (Intel i5, 8GB RAM, SSD) with the application and MySQL database running locally.

API Response Times (measured via Postman):

Endpoint	Avg Response Time	Notes
POST /api/students/login	180ms	Includes bcrypt comparison (~100ms)
POST /api/students/register	220ms	Includes bcrypt hashing (~120ms)
GET /api/admin/dashboard	45ms	Multiple aggregate queries
GET /api/admin/students?page=1	30ms	Paginated with JOIN
POST /api/admin/allocations	55ms	Includes trigger execution
GET /api/admin/reports/occupancy	25ms	Aggregate with GROUP BY

All endpoints respond well within the 500ms target specified in the non-functional requirements. The bcrypt operations (hashing and comparison) account for the majority of the latency on authentication endpoints, which is by designâ€”the deliberate slowness is a security feature.

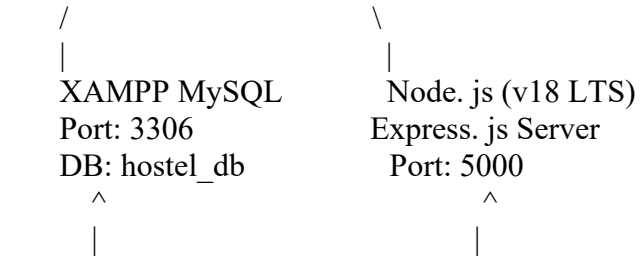
The MySQL connection pool handles concurrent requests efficiently, with no connection-wait delays observed during multi-tab testing (simulating 5 simultaneous users). The pool’s default limit of 10 connections provides ample headroom for the institution’s expected user base of 200-500 students and 5-10 administrators.

CHAPTER 9:

DEPLOYMENT AND MAINTENANCE

9.1 Deployment Environment

The Hostel Room Allotment System is installed on a local development environment that uses XAMPP like an infrastructure platform. XAMPP contains Apache (which we primarily use for phpMyAdmin), MySQL 8. 0 as our database server, and also PHP (even though it is not being used by this project but is also present). On the other hand, our Node. js backend runs independently from the Apache server of XAMPP, listening on port 5000.
Single machine deployment structure:
Windows 10/11



9. 2 Deployment Procedure

The step by step deployment process designed for reproducibility goes as follows:
1) Set up Prerequisites
• Get Node. js (version 14 or higher, with 18 LTS being the recommended one); download from nodejs. org
• Obtain XAMPP from apachefriends. org
• Install Git (this is not obligatory but is required if you would want to clone your repository)
2) Turn On XAMPP Services
• Launch XAMPP control panel
• Activate the MySQL service (you may decide to ignore Apache since it's used only in accessing phpMyAdmin)
• Confirm whether MySQL is operational at port 3306
3) Create Your Database
• Either open up phpMyAdmin at http://localhost/phpmyadmin or just use MySQL command line
• Execute database/schema. sql in order to build the hostel_db database along with every table, index view as well as trigger
• At your discretion, run database/test_data. sql with the intention of inserting demonstration data.

Step 4: Configure Environment Variables

- Navigate to the backend/ directory
- Copy .env.example to .env
- Edit .env with the correct database credentials:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=hostel_db
DB_PORT=3306
JWT_SECRET=your_secure_secret_key_here
```

JWT_REFRESH_SECRET=your_refresh_secret_key_here
JWT_EXPIRE=24h
JWT_REFRESH_EXPIRE=7d
BCRYPT_ROUNDS=10
PORT=5000
HOST=localhost
NODE_ENV=development

Step 5: Install Node.js Dependencies

cd backend
npm install

This installs all eleven production dependencies and three development dependencies listed in package.json.

Step 6: Start the Server
npm run dev # *Development mode (with nodemon auto-restart)*
or
npm start # *Production mode*

The server starts, tests the database connection, and begins listening on port 5000. A successful startup displays the server information banner with API endpoint URLs.

Step 7: Access the Application

- Open a web browser and navigate to the frontend index.html file
- For production deployment, configure the Express static file middleware to serve the frontend from the frontend/ directory

9. 3 Database Deployment

The database schema is deployed through one SQL script only which is found in (database/schema. sql). This executes the given operations in sequence:
1. Creates the hostel_db database using UTF-8 character encoding
2. Establishes eight tables and allocates correct data types as well as constraints to each field while assigning defaults where necessary
3. Develops performance indexes in highly demanded columns
4. Creates three database views that are meant to facilitate very complex reporting tasks
5. Develops three triggers for effective room occupancy automation
The test data script (database/test_data. sql) is meant to generate some sort of practical data containing dummy admin accounts, student information, hostel sections, available rooms, application forms, and allocation details for the purpose of demonstration and testing.
Database backup strategy:

- MySQL's mysqldump utility can be used to create full database backups:

```
mysqldump -u root hostel_db > backup_2026_03_09.sql
```

- Automated daily backups can be configured using Windows Task Scheduler or cron jobs on Linux

- Backups should be stored on a separate storage device or cloud service

9.4 Maintenance Plan

The maintenance tasks that are ongoing fall into four different categories:
Corrective maintenance entails addressing any defects or bugs found while using the production platform. A lot is possible in the way of isolating bugs thanks to our well organized code base structured around separate files like middleware, controller, routes, and utility.
Troubleshooting becomes easier with logs provided at each level of the program. From this description we infer that corrective maintenance corrects errors of bugs in the software system during running time of software. It is very easy to correct errors in software due to the way the file of our application is written apart and organized. We get a chance to trace some problems by the aid of print messages on all layers of the program for easier diagnosis of bugs.
Adaptive maintenance refers to modifications done to the system so that it can work perfectly with any changes in its operational environment. An example may include upgrading of Node. js to the most recent long term version (LTS), update of npm packages just to cover up for security patches or also change of database structures as per requirements of new institutional rules and regulations.
Perfective maintenance covers improvements and performance that is very much important for maintaining high standards as indicated in user views. Take an illustration of the report generator which could have extra analysis query added onto it, or enhance notification feature on the students' section of our portal. Such are types of perfective maintenance and relate to improving certain functionalities of software such as enhancing performance of program and adding some extra features in it as demanded by users of particular systems. This is very helpful to improve and enhance functions of systems and software through new designs and also for system to possess high performance and reliability by the aid of necessary changes as requested by the end users.
Preventive maintenance involves carrying out routine maintenance which maintains the stability and consistency of the system.
This can be illustrated in the following ways:

- Monthly scanning of npm audit reports for potential security threats within library dependencies
- Quarterly optimization of databases through the use of ANALYZE TABLE and OPTIMIZE TABLE commands to keep database queries fast
- Reviewing rate limiting limits twice annually, basing them on real-world usage data and patterns
- Rotation of JWT secret keys annually

When we talk about scalability strategy for now the system runs on one server alone to serve one educational institution but what is more is that the design is adaptable for these scalable strategies:
Vertically Scaling increasing the CPU cores and RAM available to Node.js server and MySQL database is possible. Also an increase of the innodb buffer pool size of MySQL would enable caching of more data in the RAM and there would be allocation of more heap memory to Node. js through command line argument –max-old-space-size. Hence both vertically scaled servers have the ability to take up much more data in their buffers.
By going horizontal on application tier and then stateless APIs become key components in supporting such scaling scenarios. Since JWT authentication does not require server session storage statelessness of Node. js was achieved, thus back-end server instances can run behind a load balancer such as Nginx with no need to alter any part of code and they will all share one MySQL database.
Database scaling: For bigger establishments with more data and heavy load of transactions, MySQL replication is another technique of distributing reads across many replica servers while only write

operation goes to main or primary server. Our web applications being known for providing dashboards and other reports with large amounts of data are very best candidates for this approach and will therefore benefit greatly from this kind of replication strategy where only reads are taken out on some other servers not the ones taking writes of data in transaction log.

Caching is yet another technique used in making it possible for our system to perform well even with lots of clients as it is designed to cut down the traffic of database queries for example through data from Redis cache of frequently requested data like statistics of various rooms and data about free time in those rooms as there are very few database operations after that.

CHAPTER 10:

CONCLUSION AND FUTURE SCOPE

10.1 Project Conclusion

The Hostel Room Allotment System aimed at addressing real problems concerning the inefficiency and lack of clarity in the traditional methods used in managing hostel rooms in educational institutions. In the end, we achieved our target of having a fully operational web application that streamlines the whole allocation process for new students starting with student registration, submission of accommodation requests and approval of admissions and after that assigns them with rooms and reports on who occupies what and when.

In developing the application, the process itself was very important as the end product. Working out the database structure made us think deeply about the relationships among entities, cardinality and data integrity requirements. Creating a RESTful API exposed the significance of having constant response formats and appropriate HTTP status codes as well as the multi-layered middleware architecture facilitated by Express. js. We learned about non-stateful authentication including JWT with practice in managing life cycle of tokens and the different implications of session based and token based systems. Including input validation rules in every endpoint drove home the point of the fact that the system becomes vulnerable to maliciously or accidentally entered wrong data when inputs are trusted without scrutiny.

It is quite evident that despite being an academic project, the system turned out to be practically useful. It follows up with the main tasks that a head of hostel encounters every day, provides the openness which students need all the time and generates the necessary reports for decision makers within the institution.

10. 2 Achievements

Now looking back to what we stipulated as project goals in chapter one, we have realized the following:

1. The whole stack of a web application was developed. The system is made up of a responsive Bootstrap 5 frontend and a structured MySQL database while the Express. js REST API serves as the backend; all these components work perfectly together to give a complete application.
2. Implemented access control based on roles. There are three clear roles like student, admin and super admin and they have different portals whereby authorization at the level of endpoints is very granular so as to make sure each user just accesses features relevant to his role.
3. A normalized database with 8 tables. Schema accurately models all related entities and relationships with use of foreign keys, unique constraints, ENUM types, computed fields, views and triggers ensuring the integrity of data right at the database level.
4. Protected the security of API with more than one layer of protection. The safety of JWT token, hash passwords using bcrypt, usage of security-related headers with the help of Helmet, enable CORS restrictions, employ rate limiters and finally use express-validator for input validation together represent good defense-in-depth security.
5. Developed dynamic dashboards in real time. Both student and admin dashboards aggregate and present current data from the database so it is possible to see some essential parameters in a few seconds.
6. Provided online services for students. In the end the student profile can be registered, apply for hostel rooms, get to know the current status of room allotment, check allocations to themselves and also update their respective information without the need of physical visits to hostel offices.
7. Included reporting functionalities. The system contains reports of students by specific blocks and data regarding students' enrollment in particular years, studies of certain gender and by course is accessible for administrators on demand.
8. Ensuring data integrity

The database triggers ensure that there is a proper record of all room occupancies and also with unique

constraints preventing any overlapping allocations. Transactional support is also there in order to safeguard multi-stage transactions.

10. 3 Limitations

Despite being able to handle everything within the given parameters, this particular release does have some limitations:

1. Absence of email notifications

It becomes incumbent upon students to actively log into the website so they can follow up on their application status. In fact, the system does not send out any email or even SMS alerts at such stages as processing of application and giving out of accommodation.

2. Lack of provision for uploading documents

Students cannot upload required supplementary documentation (ID proof, medical certificates etc.) on filling out the form. Verifications regarding eligibility have to be carried out via other means by the administration.

3. Absence of integrated online payment

In our present system, all payments are just manually handled by the admin. We don't have any integration set up in place with payment gateways meant for paying rents online.

4. A single institution concept is used

Our current database setup does not accommodate more than one tenant. Therefore, setting it up for use in different schools will require us to have different databases for them.

5. There is no stand-alone mobile app

Although the site layout is adaptable and viewable on mobile devices via internet browsers, we are missing out on native mobile apps which would offer much better user experience and some exclusive features like sending out push messages and storing information offline.

6. Limited analysis capacity

The reporting feature that we have gives us some basic data summaries and nothing beyond that– no trending analysis, no prediction for probable future utilization, nor any downloadable format to save your reports in PDF or Excel.

7. No automated allocation of rooms

Room assignment is done manually without any help from technology. Our admin should personally see through every student's request going through a very laborious process of looking out for vacant accommodations available matching to each applicant's requirements before making those tough decisions all on his own without the system helping him out with suggestions or automatic allocation based on individual choices and what is available at any moment.

10. 4 Enhancements in future plans

Below are some suggested or already considered improvements we intend to include in subsequent versions of the software:

1. An email/SMS notification system

We should include in our system an automated alert notification system by integrating Node. js library for email notifications (Nodemailer) and SMS APIs from Twilio. Such integration would provide immediate alerts to students at key milestones within the process such as the completion of their application, allotment of a room, and nearing of deadlines for making payments. That way students wouldn't need to keep checking on our portal repeatedly for such updates.

2. Uploading and managing files

Adding support for uploading files using Multer middleware would allow students to attach identity documents, medical certificates, among others in relation to their applications. All uploaded files will be saved in the server; meanwhile relevant links referencing these files are kept in our database.

3. Integration with online payment systems

Integrating Razorpay, Stripe or PayPal etc. would offer an opportunity for the students to make their payments for hostel fees on-line through our portal and get instant payment confirmations which will further facilitate updation of payment status in corresponding room allocations.

4. An intelligent algorithm of room allocation

To ease the task of our admin especially when there is heavy intake, we plan on developing a suitable smart room assignment algorithm that will look into the choices of every applicant (in respect to block, preferred type of room), satisfies necessary conditions (of eligibility e. g., respective years, gender and block compatible) as well as room availability in suggesting or auto-allocating of rooms accordingly.

5. Mobile Native Application

React Native or Flutter can be employed in order to create both Android and iOS applications. Push notifications are supported, there is also offline data caching available and what about a native user experience?

6. Advanced Analytics Dashboard

Add historical trend charts such as occupancy over time; demand forecasting for example predicted applications for next semester and revenue analytics. Use Chart. js or D3. js in designing interactive visualizations.

7. Multi-Institution Support

Redesign the schema in order to accommodate multiple tenants i. e., enable multiple institutions to utilize a common deployment but keep their data separate. Each establishment has to have its particular administration accounts, blocks, rooms as well as student pools all isolated.

8. Room Swap and Transfer Requests

Allow the students to make requests concerning room changes like exchange and transfer that needs an administration approval. The usual case when students change houses in the middle of semester is this scenario that it attends to.

9. Maintenance Request System

Enable the students in reporting any kind of problems concerning room maintenance on broken furniture, plumbing problem, and electrical fault through our website whereby tracking of request status and resolution is monitored.

10. Export and Print Functionality

Develop the capability of exporting the generated reports into PDF or Excel formats, while making possible the creation of printable letters of room assignment and receipt of students.

11. Real-time Chat Support

Integrate into our platform a real-time communication tool with help of WebSockets which enables easy and efficient direct conversation between students of the hostel and office, and thus reduces requirements of going to offices or calling them up.

12. Biometric or QR-Code Based Check-In

Implement a system of verification where students confirm on whether they occupy certain rooms by means of biometric scanning or reading out of QR codes at entry points of the hostel.

REFERENCES

- Tilkov, S. and Vinoski, S. (2010). "Node.js: Using JavaScript to Build High-Performance Network Programs." *IEEE Internet Computing*, vol. 14, no. 6, pp. 80-83.
- Cantelon, M., Harter, M., Holowaychuk, T.J. and Rajlich, N. (2014). *Node.js in Action*. Manning Publications, Shelter Island, NY.
- Brown, E. (2019). *Web Development with Node and Express: Leveraging the JavaScript Stack*. 2nd Edition. O'Reilly Media, Sebastopol, CA.
- Wexler, J. (2019). *Get Programming with Node.js*. Manning Publications, Shelter Island, NY.
- Elmasri, R. and Navathe, S.B. (2015). *Fundamentals of Database Systems*. 7th Edition. Pearson Education, Boston, MA.
- Schwartz, B., Zaitsev, P. and Tkachenko, V. (2012). *High Performance MySQL: Optimization, Backups, and Replication*. 3rd Edition. O'Reilly Media, Sebastopol, CA.
- Jones, M., Bradley, J. and Sakimura, N. (2015). "JSON Web Token (JWT)." *RFC 7519*, Internet Engineering Task Force (IETF).
- Fielding, R.T. (2000). "Architectural Styles and the Design of Network-Based Software Architectures." Doctoral dissertation, University of California, Irvine.
- Provos, N. and Mazieres, D. (1999). "A Future-Adaptable Password Scheme." *Proceedings of the USENIX Annual Technical Conference*, Monterey, CA, pp. 81-91.
- Masse, M. (2011). *REST API Design Rulebook*. O'Reilly Media, Sebastopol, CA.
- MySQL Documentation Team (2024). "MySQL 8.0 Reference Manual." Oracle Corporation. Available at: https://dev.mysql.com/doc/refman/8.0/en/
- Express.js Contributors (2024). "Express.js Guide." Available at: https://expressjs.com/en/guide/routing.html
- Node.js Foundation (2024). "Node.js Documentation." Available at: https://nodejs.org/en/docs/
- Otto, M. and Thornton, J. (2023). "Bootstrap 5 Documentation." Available at: https://getbootstrap.com/docs/5.3/
- OWASP Foundation (2021). "OWASP Top Ten Web Application Security Risks." Available at: https://owasp.org/www-project-top-ten/
- Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004). "Design Science in Information Systems Research." *MIS Quarterly*, vol. 28, no. 1, pp. 75-105.
- Pressman, R.S. and Maxim, B.R. (2014). *Software Engineering: A Practitioner's Approach*. 8th Edition. McGraw-Hill Education, New York, NY.
- Sommerville, I. (2015). *Software Engineering*. 10th Edition. Pearson Education, Boston, MA.
- Flanagan, D. (2020). *JavaScript: The Definitive Guide*. 7th Edition. O'Reilly Media, Sebastopol, CA.
- Duckett, J. (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley Publishing, Indianapolis, IN.

APPENDIX A: SOURCE CODE

A.1 Server Entry Point (server.js)

The main server file initializes Express, configures middleware, mounts routes, and starts listening.

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const morgan = require("morgan");
const rateLimit = require("express-rate-limit");
const path = require("path");
const { testConnection } = require("./config/database");
const studentRoutes = require("./routes/studentRoutes");
const adminRoutes = require("./routes/adminRoutes");

const app = express();
const PORT = process.env.PORT || 5000;
const HOST = process.env.HOST || "localhost";

// Security Middleware
app.use(
  helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        styleSrc: [
          "'self'",
          "'unsafe-inline'",
          "https://cdn.jsdelivr.net",
          "https://cdnjs.cloudflare.com",
        ],
        scriptSrc: [
          "'self'",
          "'unsafe-inline'",
          "https://cdn.jsdelivr.net",
          "https://cdnjs.cloudflare.com",
        ],
        imgSrc: ["'self'", "data:", "https:"],
        fontSrc: ["'self'", "https://cdnjs.cloudflare.com"],
        connectSrc: ["'self'", `http://localhost:${PORT}`],
      },
    },
    crossOriginEmbedderPolicy: false,
  }),
);

// CORS
const corsOptions = {
  origin: process.env.CORS_ORIGIN
    ? process.env.CORS_ORIGIN.split(",")
    : ["http://localhost:3000", "http://127.0.0.1:5500"],
```

```
  methods: ["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization", "X-Requested-With"],
  credentials: true,
  maxAge: 86400,
};
app.use(cors(corsOptions));

// Rate limiting
const limiter = rateLimit({
  windowMs: parseInt(process.env.RATE_LIMIT_WINDOW_MS) || 15 * 60 * 1000,
  max: parseInt(process.env.RATE_LIMIT_MAX_REQUESTS) || 100,
  message: {
    success: false,
    message: "Too many requests from this IP, please try again later.",
  },
});
app.use("/api/", limiter);

// Auth rate limiting (stricter)
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: {
    success: false,
    message: "Too many login attempts, please try again after 15 minutes.",
  },
});
app.use("/api/students/login", authLimiter);
app.use("/api/admin/login", authLimiter);

// Logging, Body Parsing
if (process.env.NODE_ENV === "development") app.use(morgan("dev"));
else app.use(morgan("combined"));

app.use(express.json({ limit: "10mb" }));
app.use(express.urlencoded({ extended: true, limit: "10mb" }));

// Health Check
app.get("/api/health", async (req, res) => {
  const dbConnected = await testConnection();
  res.status(dbConnected ? 200 : 500).json({
    success: dbConnected,
    message: dbConnected ? "Server is running" : "Database connection failed",
    timestamp: new Date().toISOString(),
    version: "1.0.0",
  });
});

// API Routes
app.use("/api/students", studentRoutes);
app.use("/api/admin", adminRoutes);

// 404 Handler
app.use((req, res) => {
  res.status(404).json({ success: false, message: "Route not found" });
```

```
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error("Error:", err);
  if (err.code === "ER_DUP_ENTRY") {
    return res.status(409).json({
      success: false,
      message: "Duplicate entry - record already exists",
    });
  }
  res.status(err.status || 500).json({
    success: false,
    message: err.message || "Internal server error",
  });
});

// Server Start
const startServer = async () => {
  try {
    const dbConnected = await testConnection();
    if (!dbConnected) {
      console.error("Failed to connect to database.");
      process.exit(1);
    }
    app.listen(PORT, HOST, () => {
      console.log(`Server running at http://${HOST}:${PORT}`);
    });
  } catch (error) {
    console.error("Server startup error:", error);
    process.exit(1);
  }
};

startServer();
module.exports = app;
```

A.2 Database Configuration (config/database.js)

```
const mysql = require("mysql2/promise");
require("dotenv").config();

const dbConfig = {
  host: process.env.DB_HOST || "localhost",
  user: process.env.DB_USER || "root",
  password: process.env.DB_PASSWORD || "",
  database: process.env.DB_NAME || "hostel_db",
  port: process.env.DB_PORT || 3306,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
  connectTimeout: 10000,
  acquireTimeout: 10000,
  namedPlaceholders: true,
  dateStrings: true,
```

```
  debug: process.env.DB_DEBUG === "true",
};

const pool = mysql.createPool(dbConfig);

const testConnection = async () => {
  try {
    const connection = await pool.getConnection();
    console.log("Database connection established successfully");
    connection.release();
    return true;
  } catch (error) {
    console.error("Database connection failed:", error.message);
    return false;
  }
};

const query = async (sql, params = []) => {
  try {
    const [results] = await pool.execute(sql, params);
    return results;
  } catch (error) {
    console.error("Query Error:", error.message);
    throw error;
  }
};

const transaction = async (callback) => {
  const connection = await pool.getConnection();
  try {
    await connection.beginTransaction();
    const result = await callback(connection);
    await connection.commit();
    return result;
  } catch (error) {
    await connection.rollback();
    throw error;
  } finally {
    connection.release();
  }
};

module.exports = { pool, query, transaction, testConnection };
```

A.3 Authentication Middleware (middleware/auth.js)

```
const jwt = require("jsonwebtoken");

const verifyToken = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(401).json({
        success: false,
        message: "Access denied. No token provided.",
```

```
      });
    }
    const token = authHeader.substring(7);
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = {
      id: decoded.id,
      userType: decoded.userType,
      email: decoded.email,
      role: decoded.role || null,
    };
    next();
  } catch (error) {
    if (error.name === "TokenExpiredError") {
      return res.status(401).json({
        success: false,
        message: "Token has expired.",
      });
    }
    return res.status(401).json({
      success: false,
      message: "Invalid token.",
    });
  }
};

const verifyStudent = (req, res, next) => {
  if (req.user.userType !== "student") {
    return res.status(403).json({
      success: false,
      message: "Student access required.",
    });
  }
  next();
};

const verifyAdmin = (req, res, next) => {
  if (req.user.userType !== "admin") {
    return res.status(403).json({
      success: false,
      message: "Admin access required.",
    });
  }
  next();
};

const verifySuperAdmin = (req, res, next) => {
  if (req.user.userType !== "admin" || req.user.role !== "super_admin") {
    return res.status(403).json({
      success: false,
      message: "Super admin access required.",
    });
  }
  next();
};
```

```
module.exports = { verifyToken, verifyStudent, verifyAdmin, verifySuperAdmin };
```

A.4 Utility Helpers (utils/helpers.js)

```
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const crypto = require("crypto");

// Password Hashing
const hashPassword = async (password) => {
  const saltRounds = parseInt(process.env.BCRYPT_ROUNDS) || 10;
  return await bcrypt.hash(password, saltRounds);
};

const comparePassword = async (password, hashedPassword) => {
  return await bcrypt.compare(password, hashedPassword);
};

// JWT Token Generation
const generateTokens = (user) => {
  const payload = {
    id: user.id,
    userType: user.userType,
    email: user.email,
    role: user.role || null,
  };
  return {
    accessToken: jwt.sign(payload, process.env.JWT_SECRET, {
      expiresIn: process.env.JWT_EXPIRE || "24h",
    }),
    refreshToken: jwt.sign(payload, process.env.JWT_REFRESH_SECRET, {
      expiresIn: process.env.JWT_REFRESH_EXPIRE || "7d",
    }),
  };
};

// Response Utilities
const sendSuccess = (
  res,
  message = "Success",
  data = null,
  statusCode = 200,
) => {
  const response = { success: true, message };
  if (data !== null) response.data = data;
  return res.status(statusCode).json(response);
};

const sendError = (res, message = "Error", statusCode = 500, errors = null) => {
  const response = { success: false, message };
  if (errors !== null) response.errors = errors;
  return res.status(statusCode).json(response);
```

```
};

// Pagination
const createPagination = (page, limit, total) => {
  const totalPages = Math.ceil(total / limit);
  return {
    currentPage: page,
    totalPages,
    totalItems: total,
    itemsPerPage: limit,
    hasNextPage: page < totalPages,
    hasPrevPage: page > 1,
  };
};

const getPaginationParams = (req) => {
  const page = parseInt(req.query.page) || 1;
  const limit = Math.min(parseInt(req.query.limit) || 10, 100);
  const offset = (page - 1) * limit;
  return { page, limit, offset };
};

// Date Utilities
const formatDate = (date) => {
  if (!date) return null;
  return new Date(date).toISOString().split("T")[0];
};

// Object Utilities
const omit = (obj, keys) => {
  return Object.fromEntries(
    Object.entries(obj).filter(([key]) => !keys.includes(key)),
  );
};

module.exports = {
  hashPassword,
  comparePassword,
  generateTokens,
  sendSuccess,
  sendError,
  createPagination,
  getPaginationParams,
  formatDate,
  omit,
};
```

A.5 Database Schema (database/schema.sql)

```
CREATE DATABASE IF NOT EXISTS hostel_db
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE hostel_db;

-- Admin table
CREATE TABLE admin (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    role ENUM('super_admin', 'admin') DEFAULT 'admin',
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    last_login TIMESTAMP NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Students table
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(20) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE,
    gender ENUM('male', 'female', 'other') NOT NULL,
    phone VARCHAR(20),
    address TEXT,
    course VARCHAR(100) NOT NULL,
    department VARCHAR(100),
    year_of_study INT NOT NULL,
    emergency_contact_name VARCHAR(100),
    emergency_contact_phone VARCHAR(20),
    profile_image VARCHAR(255),
    is_active BOOLEAN DEFAULT TRUE,
    email_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    last_login TIMESTAMP NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Hostel Blocks table
CREATE TABLE hostel_blocks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    block_name VARCHAR(50) NOT NULL UNIQUE,
    block_code VARCHAR(10) NOT NULL UNIQUE,
    gender_type ENUM('male', 'female', 'mixed') NOT NULL,
    description TEXT,
    total_floors INT DEFAULT 1,
    warden_name VARCHAR(100),
    warden_contact VARCHAR(20),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Rooms table
CREATE TABLE rooms (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_number VARCHAR(20) NOT NULL,
    block_id INT NOT NULL,
    floor_number INT DEFAULT 1,
    room_type ENUM('single', 'double', 'triple', 'quad') DEFAULT 'double',
    capacity INT NOT NULL DEFAULT 2,
    occupied_beds INT DEFAULT 0,
    available_beds INT GENERATED ALWAYS AS (capacity - occupied_beds) STORED,
    room_status ENUM('available', 'full', 'maintenance', 'reserved')
        DEFAULT 'available',
    facilities TEXT,
    monthly_rent DECIMAL(10, 2) DEFAULT 0.00,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (block_id) REFERENCES hostel_blocks(id) ON DELETE CASCADE,
    UNIQUE KEY unique_room_block (room_number, block_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Room Applications table
CREATE TABLE room_applications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    preferred_block_id INT,
    preferred_room_type ENUM('single','double','triple','quad','any') DEFAULT 'any
',
    application_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    preferred_start_date DATE,
    duration_months INT DEFAULT 12,
    special_requirements TEXT,
    status ENUM('pending','approved','rejected','waitlisted','cancelled')
        DEFAULT 'pending',
    admin_remarks TEXT,
    processed_by INT,
    processed_at TIMESTAMP NULL,
    FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
    FOREIGN KEY (preferred_block_id) REFERENCES hostel_blocks(id) ON DELETE SET NU
LL,
    FOREIGN KEY (processed_by) REFERENCES admin(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Room Allocations table
CREATE TABLE room_allocations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    room_id INT NOT NULL,
    application_id INT,
    bed_number INT DEFAULT 1,
    allocation_date DATE NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    rent_amount DECIMAL(10, 2) NOT NULL,
    security_deposit DECIMAL(10, 2) DEFAULT 0.00,
    payment_status ENUM('pending','partial','paid','refunded') DEFAULT 'pending',
    allocation_status ENUM('active','expired','terminated','extended')
```

```
        DEFAULT 'active',
    termination_reason TEXT,
    allocated_by INT NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
    FOREIGN KEY (application_id) REFERENCES room_applications(id) ON DELETE SET NU
LL,
    FOREIGN KEY (allocated_by) REFERENCES admin(id) ON DELETE CASCADE,
    UNIQUE KEY unique_student_active (student_id, allocation_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Payments table
CREATE TABLE payments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    allocation_id INT NOT NULL,
    student_id INT NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    payment_type ENUM('rent','security_deposit','fine','other') DEFAULT 'rent',
    payment_method ENUM('cash','card','bank_transfer','online') NOT NULL,
    transaction_id VARCHAR(100),
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status ENUM('pending','completed','failed','refunded') DEFAULT 'pending',
    remarks TEXT,
    recorded_by INT,
    FOREIGN KEY (allocation_id) REFERENCES room_allocations(id) ON DELETE CASCADE,
    FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
    FOREIGN KEY (recorded_by) REFERENCES admin(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Audit Logs table
CREATE TABLE audit_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_type ENUM('student', 'admin') NOT NULL,
    user_id INT NOT NULL,
    action VARCHAR(100) NOT NULL,
    table_name VARCHAR(50),
    record_id INT,
    old_values JSON,
    new_values JSON,
    ip_address VARCHAR(45),
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Performance Indexes
CREATE INDEX idx_students_email ON students(email);
CREATE INDEX idx_students_student_id ON students(student_id);
CREATE INDEX idx_rooms_block ON rooms(block_id);
CREATE INDEX idx_rooms_status ON rooms(room_status);
CREATE INDEX idx_allocations_student ON room_allocations(student_id);
CREATE INDEX idx_allocations_room ON room_allocations(room_id);
CREATE INDEX idx_allocations_status ON room_allocations(allocation_status);
CREATE INDEX idx_applications_student ON room_applications(student_id);
CREATE INDEX idx_applications_status ON room_applications(status);
```

```
-- Database Triggers
DELIMITER //

CREATE TRIGGER trg_after_allocation_insert
AFTER INSERT ON room_allocations
FOR EACH ROW
BEGIN
    IF NEW.allocation_status = 'active' THEN
        UPDATE rooms
        SET occupied_beds = occupied_beds + 1,
            room_status = CASE
                WHEN occupied_beds + 1 >= capacity THEN 'full'
                ELSE 'available'
            END
        WHERE id = NEW.room_id;
    END IF;
END//

CREATE TRIGGER trg_after_allocation_update
AFTER UPDATE ON room_allocations
FOR EACH ROW
BEGIN
    IF OLD.allocation_status = 'active' AND NEW.allocation_status != 'active' THEN
        UPDATE rooms
        SET occupied_beds = GREATEST(occupied_beds - 1, 0),
            room_status = 'available'
        WHERE id = OLD.room_id;
    ELSEIF OLD.allocation_status != 'active' AND NEW.allocation_status = 'active'
THEN
        UPDATE rooms
        SET occupied_beds = occupied_beds + 1,
            room_status = CASE
                WHEN occupied_beds + 1 >= capacity THEN 'full'
                ELSE 'available'
            END
        WHERE id = NEW.room_id;
    END IF;
END//

CREATE TRIGGER trg_after_allocation_delete
AFTER DELETE ON room_allocations
FOR EACH ROW
BEGIN
    IF OLD.allocation_status = 'active' THEN
        UPDATE rooms
        SET occupied_beds = GREATEST(occupied_beds - 1, 0),
            room_status = 'available'
        WHERE id = OLD.room_id;
    END IF;
END//

DELIMITER ;
```

A.6 Environment Variable Configuration (.env)

```
# Database Configuration
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=hostel_db
DB_DEBUG=false

# JWT Configuration
JWT_SECRET=hostel_room_allotment_super_secret_key_2024
JWT_REFRESH_SECRET=hostel_room_allotment_refresh_secret_key_2024
JWT_EXPIRE=24h
JWT_REFRESH_EXPIRE=7d

# Server Configuration
PORT=5000
HOST=localhost
NODE_ENV=development

# Security
BCRYPT_ROUNDS=10
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100

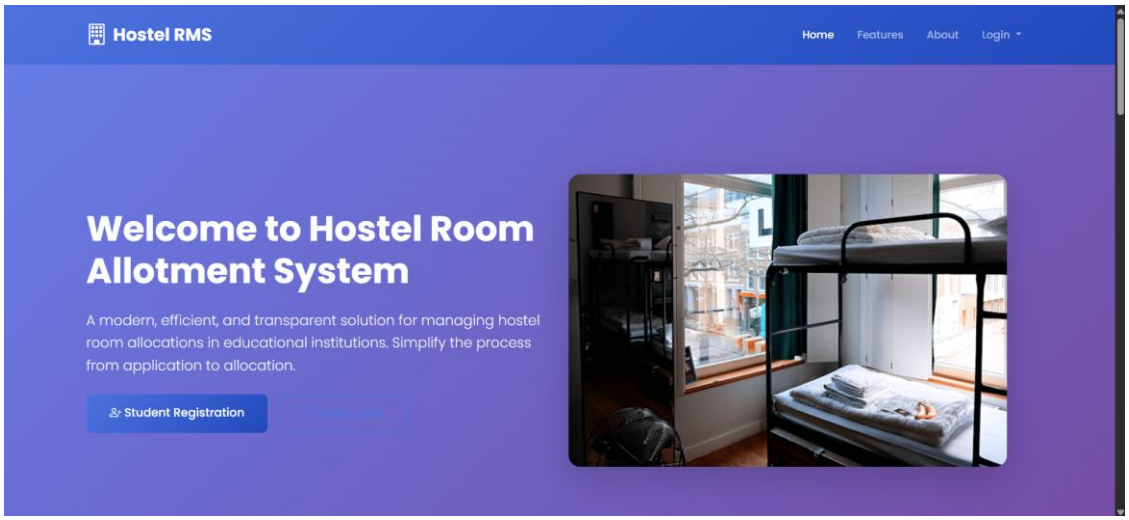
# CORS
CORS_ORIGIN=http://localhost:3000,http://127.0.0.1:5500
```

APPENDIX B: APPLICATION SCREENSHOTS

This appendix presents screenshots of the Hostel Room Allotment System’s user interface, demonstrating the visual design and functional layout of key pages.

B.1 Home Page

The landing page serves as the public entry point to the system. It features a hero section with a welcome message and registration call-to-action, animated statistics counters, a feature showcase with six key capabilities, role-specific portal cards for student and admin access, an about section explaining the system’s purpose, and a comprehensive footer with navigation links and contact information. The design uses the Poppins font family, Bootstrap 5 components, and a blue-green color scheme for a modern, professional appearance.

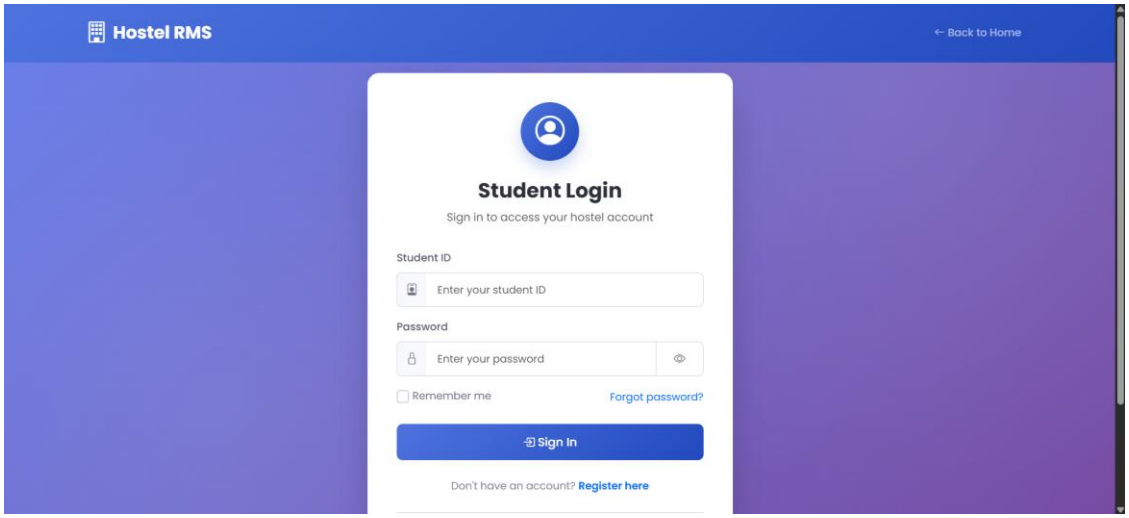


Home Page Screenshot

Figure B.1: Home Page â€“ Landing page of the Hostel Room Allotment System showing the hero section, feature cards, and navigation

B.2 Student Login Page

The student login page provides a clean, focused authentication interface. Students enter their Student ID and password to access the student portal. The page includes a link to the registration page for first-time users and a “Back to Home” navigation option. The form implements client-side validation for required fields and displays error messages from the server (invalid credentials, rate limiting) within the page context.

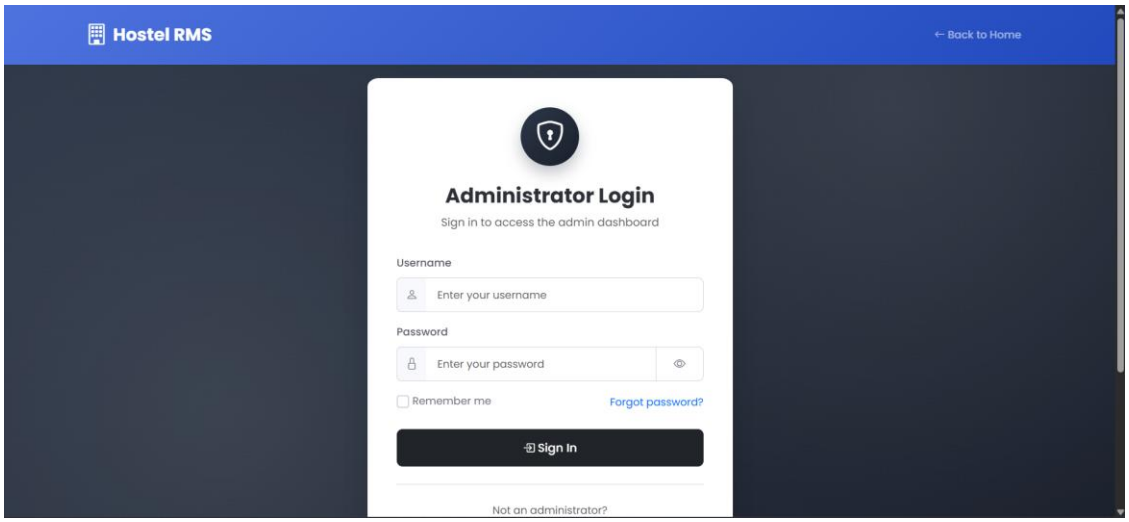


Student Login Page Screenshot

Figure B.2: Student Login Page â€“ Authentication form for student portal access

B.3 Admin Login Page

The admin login page follows the same design language as the student login but uses a darker color scheme to visually distinguish the administrative portal. Administrators enter their username and password. The page is protected by a stricter rate limiter that allows only five login attempts per fifteen-minute window, providing additional security for the privileged admin accounts.



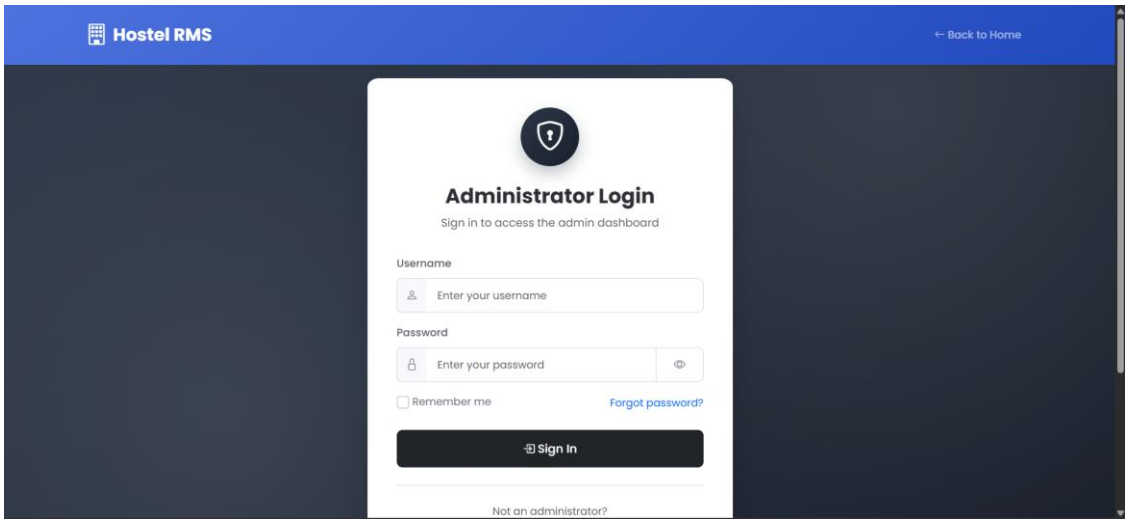
Admin Login Page Screenshot

Figure B.3: Admin Login Page â€“ Authentication form for administrative portal access

Figure B.2: Student Login Page â€“ Authentication form for student portal access

B.3 Admin Login Page

The admin login page follows the same design language as the student login but uses a darker color scheme to visually distinguish the administrative portal. Administrators enter their username and password. The page is protected by a stricter rate limiter that allows only five login attempts per fifteen-minute window, providing additional security for the privileged admin accounts.



Admin Login Page Screenshot

Figure B.3: Admin Login Page â€“ Authentication form for administrative portal access